GOOGLE | 1POINT3ACRES, PRACHUB | TOPIC 7 OF 10 | COMMON TOPIC

Binary Search and Sorting

12 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	9	2
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Binary Search and Sorting | 12 questions | tap a title to jump to it

BINARY SEARCH (6)

E	1. Check an Interval Against Existing Intervals for Conflicts	PracHub	2026-08-08
M	2. Cutting Ribbons (LC 1891)	1Point3Acres	2026-08-16
M	3. First Bad Version with Parallel Search Follow-up	1Point3Acres	2026-06-15
M	4. Find median of merged RLE arrays	PracHub	2026-01-06
M	5. Solve Banana Speed and Interval Merge	PracHub	2025-01-27
H	6. Find Maximum Envelope Nesting	PracHub	2026-03-18

SORTING (6)

M	7. Progressive Array Partitioning	1Point3Acres	2026-08-11
M	8. Streaming Points: Emit K Within Distance	1Point3Acres	2026-06-23
M	9. Find Containing Range	PracHub	2026-06-04
M	10. Check if all substrings are anagrams of words	PracHub	2026-03-11
M	11. Find shared objects across two log files	PracHub	2025-11-05
H	12. Find safe travel intervals between planet influences	PracHub	2025-11-22

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Binary search

EASY | IN APP LC 729 | PracHub | Technical Screen | 1 of 12

Check an Interval Against Existing Intervals for Conflicts

POSTED 2026-08-08 | 73 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 26.5 | 30 DAYS 0 | PRIORITY 132/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Binary search

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #729 My Calendar I

LeetCode match: LC 729 My Calendar I (inferred from overview)

Posted on PracHub: 2026-08-08 (updated 2026-08-29)

Original: https://prachub.com/coding-questions/check-an-interval-against-existing-intervals-for-conflicts

QUESTION

Overview: Check whether one half-open candidate interval conflicts with any interval in a large unsorted collection. Respect strict start and end semantics so ranges that only touch at an endpoint do not conflict.

APPROACH

Half-open intervals [s,e) conflict iff new.start < e and s < new.end (touching endpoints do not conflict). One-off check on an unsorted collection is a single O(n) scan; for repeated queries keep a sorted structure (TreeMap/balanced BST or sorted list + bisect) and check only the neighbors. O(log n) per query.

HOW TO THINK ABOUT IT

Two half-open intervals [s1, e1) and [s2, e2) overlap exactly when s1 < e2 and s2 < e1. If the booked intervals are kept sorted and disjoint, only the neighbours at the insertion point can conflict, so a binary search replaces the scan.

Basis: LeetCode 729, My Calendar I (booking succeeds only when the new half-open interval overlaps nothing), and 731 for double booking.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
booked = []
def book(s, e):
    for s2, e2 in booked:
        if s < e2 and s2 < e:
            return False  # overlap
    booked.append((s, e))
    return True
```

BRUTE FORCE

TIME	SPACE
O(n) per booking	O(n)

WHY IT IS SLOW

Each booking scans all earlier bookings, O(n). With the intervals sorted, binary search finds the only two neighbours that could conflict.

OPTIMAL SOLUTION · Sorted starts with bisect: check only two neighbours

TIME	SPACE
O(log n) search, O(n) list insert	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Allow double booking but never triple booking (LeetCode 731).

Keep a difference map (+1 at start, -1 at end) and sweep it in sorted order; if the running count ever exceeds 2, undo the change and reject. O(n log n) per booking, or use a segment tree for faster.

FOLLOW-UP 2 Report the maximum number of overlapping bookings after each booking (LeetCode 732).

The same difference map, taking the maximum running count in the sweep. A segment tree with lazy range increments gives O(log n) per booking.

FOLLOW-UP 3 Closed intervals instead of half-open.

Then touching endpoints conflict, so the tests become <= instead of <. Say it, and keep the choice in one comparison.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check an Interval Against Existing
# Intervals for Conflicts
#
# Check whether one half-open candidate interval conflicts
# with any interval in a large unsorted collection.
# Respect strict start and end semantics so ranges that
# only touch at an endpoint do not conflict.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Allow double booking but never triple booking
# (LeetCode 731).
# 2. Report the maximum number of overlapping bookings
# after each booking (LeetCode 732).
```

```
# 3. Closed intervals instead of half-open.
#
# INTERVIEW NOTES
# Two half-open intervals [s1, e1) and [s2, e2) overlap
# exactly when s1 < e2 and s2 < e1. If the booked
# intervals are kept sorted and disjoint, only the
# neighbours at the insertion point can conflict, so a
# binary search replaces the scan.
#
# BRUTE FORCE (say it first): Scan every booked interval
# Compare the new interval with all existing ones using
# the overlap test. Correct and simple, but each booking
# costs time proportional to the number of bookings. Time
# O(n) per booking. Space O(n).


from bisect import bisect_right
from collections import defaultdict


# BRUTE FORCE (say it first): Scan every booked interval |
# time O(n) per booking
class CalendarBrute:
    def __init__(self):
        self.booked = []

    def book(self, start, end):
        if any(
            start < e and s < end
            for s, e in self.booked
        ):
            return False
```

```
        self.booked.append((start, end))
        return True


# BASE QUESTION (optimal): Sorted starts with bisect:
# check only two neighbours
# Idea: Keep the booked intervals sorted by start.
class Calendar:
    """A sorted calendar: O(log n) per check."""

    def __init__(self):
        self.starts, self.ends = [], []

    def book(self, start, end):
        i = bisect_right(self.starts, start)
        # the booking before must end by `start`
        if i > 0 and self.ends[i - 1] > start:
            return False
        # the booking after must start at or after `end`
        if i < len(self.starts) and self.starts[i] < end:
            return False
        self.starts.insert(i, start)
        self.ends.insert(i, end)
        return True


# FOLLOW-UP 1: Allow double booking but never triple
# booking (LeetCode 731).
# Answer: Keep a difference map (+1 at start, -1 at end)
# and sweep it in sorted order; if the running count ever
# exceeds 2, undo the change and reject. O(n log n) per
# booking, or use a segment tree for faster.
class DoubleBooking:
    """LeetCode 731: two overlaps are fine, three are not.
    LeetCode 732: also report the largest overlap."""

    def __init__(self, limit=2):
        self.delta = defaultdict(int)
        self.limit = limit
```

```
    def peak(self):
        # sweep the +1 / -1 changes in time order
        run = best = 0
        for t in sorted(self.delta):
            run += self.delta[t]
```

```
            best = max(best, run)
        return best

    def book(self, start, end):
        self.delta[start] += 1
        self.delta[end] -= 1
        if self.peak() > self.limit:
            # undo the change and reject the booking
            self.delta[start] -= 1
            self.delta[end] += 1
            return False
        return True


# HELPERS for the checks below
def conflicts(candidate, intervals):
    # half-open [start, end): touching ends do NOT
    # conflict
    s, e = candidate
    return any(a < e and s < b for a, b in intervals)


if __name__ == "__main__":
    taken = [(10, 20), (30, 40)]
    assert conflicts((15, 25), taken)
    assert not conflicts((20, 30), taken)
    assert conflicts((19, 31), taken)
    # sorted calendar
    c = Calendar()
    assert c.book(10, 20) and c.book(20, 30)
    assert not c.book(15, 25) and c.book(5, 10)
    # double booking is allowed, triple booking is not
    d = DoubleBooking()
    assert d.book(10, 20) and d.book(15, 25)
    assert not d.book(17, 18)
    assert d.peak() == 2
    # limit 1 is the plain no-overlap calendar
    one = DoubleBooking(limit=1)
    assert one.book(1, 5) and not one.book(4, 6)
```

FUNCTION AND TEST CASES

Signature

```
def hasIntervalConflict(candidate: list[int], intervals: l
ist[list[int]]) -> bool:
```

Sample call

```
hasIntervalConflict([4,7], [[1,3],[7,9]])
```

Test cases

Input	Expected
([4,7],[[1,3],[7,9]])	False
([4,7],[[1,5],[8,9]])	True

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Binary search

Site tags: array, binary-search, complexity-analysis, medium

MEDIUM | 1Point3Acres | onsite-coding | 2 of 12

Cutting Ribbons (LC 1891)

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **ALL TIME 19.6** | **30 DAYS 0** | **PRIORITY 147/228**

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Binary search

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1891 Cutting Ribbons

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/cutting-ribbons-max-equal-length

QUESTION

LC 1891 Cutting Ribbons: given wood lengths and K, maximize the equal piece length that yields at least K pieces. Follow-ups cover the search bound, floating-point lengths, and why complexity depends on the value range rather than the number of elements.SWEarraybinary-searchcomplexity-analysismedium FrequencySingle reportLast asked2026-08-16Stageonsite-coding

APPROACH

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether piece lengths must be integers, what to return if even length 1 fails (0), and the sizes of the array and k.

BRUTE FORCE TO SAY OUT LOUD

Try every length from the maximum down to 1 and count pieces. O(n * max).

OPTIMAL IDEA

Feasibility is monotone: if you can cut k pieces of length L you can for any smaller length. Binary search L over [1, max]: count = sum(r // L), move right if count >= k. O(n log max). A tighter upper bound is min(max, sum // k).

DRY-RUN THESE CASES

k larger than the total length (0), one ribbon, all equal ribbons, k = 1 (answer is the max ribbon).

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Real-valued lengths (fixed number of halvings, not an epsilon loop), why complexity depends on the value range, or an unlimited number of cuts.

PITFALLS

Overflow when summing. Searching from 0 and dividing by zero.

HOW TO THINK ABOUT IT

'Largest length such that at least K pieces fit' is a maximisation with a yes/no check. Larger lengths yield fewer pieces, so the check is monotone and you can binary search the answer. The check itself is one pass: sum of ribbon // length.

Basis: LeetCode 1891, Cutting Ribbons.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(ribbons, k):
    for L in range(max(ribbons), 0, -1):
        pieces = sum(r // L for r in ribbons)
        if pieces >= k:
            return L
    return 0
```

TIME	SPACE
O(n * max)	O(1)

WHY IT IS SLOW

Tries up to max(ribbon) lengths, each costing O(n); with ribbons up to 10^9 that is a billion probes. The check is monotone, so binary search needs only log2(max) probes.

OPTIMAL SOLUTION · Binary search the answer (integer lengths)

TIME	SPACE
O(n log(max length))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the running time depend on the value range and not just on the number of ribbons?

Each probe costs O(n), and the number of probes is log2(max length) for the integer version. A huge length like 10^9 needs about 30 probes while 10 needs 4, even with the same n. For real numbers the probe count is set by the precision you promise, e.g. log2(max / epsilon).

FOLLOW-UP 2 What if the candidate length is given to you?

Then it is just the feasibility check: sum(r // L for r in ribbons) >= k, in O(n). The binary search exists only to find the best L; the check is the same function you reuse inside it.

FOLLOW-UP 3 What is a good upper bound for the search?

min(max(ribbons), sum(ribbons) // k). If the total length divided by k is smaller than the longest ribbon, no piece longer than that can ever give k pieces, which shrinks the range a little and avoids wasted probes. **Cost:** each probe is O(n); the tighter bound removes only a few of the log2(max) probes, so the total stays O(n log max).

FOLLOW-UP 4 Give the search a tighter upper bound.

No piece can be longer than the longest ribbon, and K pieces of length L need K * L total length, so L <= sum // K. Starting hi at min(max(ribbons), sum // K) saves a few probes and also returns 0 immediately when sum < K.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Cutting Ribbons (LC 1891)
#
# LC 1891 Cutting Ribbons: given wood lengths and K,
# maximize the equal piece length that yields at least K
# pieces. Follow-ups cover the search bound, floating-
# point lengths, and why complexity depends on the value
# range rather than the number of elements.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the running time depend on the value range
# and not just on the number of ribbons?
# 2. What if the candidate length is given to you?
# 3. What is a good upper bound for the search?
# 4. Give the search a tighter upper bound.
#
# INTERVIEW NOTES
# 'Largest length such that at least K pieces fit' is a
# maximisation with a yes/no check. Larger lengths yield
# fewer pieces, so the check is monotone and you can
# binary search the answer. The check itself is one pass:
# sum of ribbon // length.
#
# BRUTE FORCE (say it first): Try every length from the
# longest ribbon down
# For each length L from the maximum ribbon down to 1
# count the pieces and return the first L that gives at
# least K. Correct, but with ribbons up to 10^9 it would
# try a billion lengths. Time O(n * max). Space O(1).


# BRUTE FORCE (say it first): Try every length from the
# longest ribbon down | time O(n * max)
def max_length_brute(ribbons, k):
    for L in range(max(ribbons), 0, -1):
        if sum(r // L for r in ribbons) >= k:
            return L
    return 0
```

```
# HELPERS for the base question
def pieces(ribbons, length):
    # how many pieces of this length can we cut?
    return sum(r // length for r in ribbons)
```

```
# FOLLOW-UP 4: Give the search a tighter upper bound.
# Answer: No piece can be longer than the longest ribbon,
# and K pieces of length L need K * L total length, so L
# <= sum // K. Starting hi at min(max(ribbons), sum // K)
# saves a few probes and also returns 0 immediately when
# sum < K.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def upper_bound(ribbons, k):
    # no piece is longer than the longest ribbon, and
    # k pieces of length L need k * L total length
    return min(max(ribbons), sum(ribbons) // k)


# BASE QUESTION (optimal): Binary search the answer
# (integer lengths)
# Idea: If you can cut k pieces of length L you can cut k
# pieces of any shorter length, so feasibility is
# monotone.
def max_length(ribbons, k):
    # the best whole-number length (0 if impossible)
    lo, hi, best = 1, upper_bound(ribbons, k), 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if pieces(ribbons, mid) >= k:
            # mid works: remember it and try longer
            best, lo = mid, mid + 1
        else:
            hi = mid - 1
    return best


# VARIANT: fractional lengths
def max_length_float(ribbons, k, rounds=50):
    # lengths may be fractions: fixed rounds of halving
    lo, hi = 0.0, float(max(ribbons))
    for _ in range(rounds):
        mid = (lo + hi) / 2
        # enough pieces: the answer is at least mid
        if sum(int(r // mid) for r in ribbons) >= k:
            lo = mid
        else:
            hi = mid
    return lo
```

```
if __name__ == "__main__":
    assert max_length([9, 7, 5], 3) == 5
    assert max_length([7, 5, 9], 4) == 4
    # not enough total length: no pieces at all
    assert max_length([1, 2], 10) == 0
    # the check alone, when the length is given
    assert pieces([9, 7, 5], 5) == 3
    assert upper_bound([9, 7, 5], 3) == 7
    # fractional lengths: 3 pieces from one ribbon of 1
    assert abs(max_length_float([1], 3) - 1 / 3) < 1e-9
    assert max_length_float([1], 0 + 2) > 0.49
```

NOTES FROM CANDIDATES

The interviewer allowed the candidate to abandon an initial greedy/sorting direction, then unlocked the problem with one question: what changes if a candidate length is supplied?
Expect the follow-ups, not the base implementation, to consume most of the round.
For the integer version, initialize the best answer to 0, search only positive candidate lengths, and return 0 when even length 1 cannot produce K pieces. The feasibility predicate is monotone: once a length is infeasible, every larger length is also infeasible.

PREPARATION

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Binary search

Site tags: binary-search, parallelism, concurrency, medium

MEDIUM | 1Point3Acres | phone-screen | 3 of 12

★ TOP 75: rank 72 in the Medium and Hard priority list

★ First Bad Version with Parallel Search Follow-up

ASKED 2026-06-15 | 127 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 41.1** | **30 DAYS 33** | **PRIORITY 47/228**

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Binary search

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #278 First Bad Version

LeetCode match: LC 278 First Bad Version

Frequency: Single report | Last asked: 2026-06-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/first-bad-version-parallel-search

QUESTION

Phone-screen R1 coding round. The base problem is the standard first-bad-version monotonic search — given an **`isBadVersion(v)`** oracle over versions **`1..n`**, find the first bad version with a single binary search. The graded follow-up requires parallelizing the search: partition the range into buckets and search them concurrently (bucket + binary search), with working code expected.SWEbinary-searchparallelismconcurrencymediumFrequencySingle reportLast asked2026-06-15Stagephone-screen

APPROACH

Write the single-threaded binary search first and verify it on small n.
Then implement the bucketed parallel version with real concurrency primitives (Python concurrent.futures.ThreadPoolExecutor, or Java ExecutorService / CompletableFuture): split [1, n] into k buckets, probe the k boundaries concurrently, pick the bucket where the predicate flips, and binary-search within it. Practice getting this to compile and run in under 20 minutes.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the oracle is (isBadVersion is monotone), what concurrency primitive to use (ThreadPoolExecutor, Java's ExecutorService), and which metric matters: wall-clock time or total calls.

BRUTE FORCE TO SAY OUT LOUD

Check every version from 1. O(n) calls.

INTERVIEW-DAY PLAN

OPTIMAL IDEA

Base: binary search, O(log n) calls. Parallel: each round fires up to k probes concurrently at evenly spaced points, finds the first probe that is bad, and narrows the interval to the gap between it and the previous probe. Each round shrinks the interval by k + 1, so there are about log base (k + 1) of n sequential rounds. Use a thread pool and map the probes in one call.

DRY-RUN THESE CASES

n = 1, the first version bad, the last version bad, n smaller than the number of workers.

LIKELY FOLLOW-UPS

Rate limiting, retries when a probe fails, and caching results.

PITFALLS

Mutating shared state from threads. Assuming more threads always means fewer total calls (they increase it).

HOW TO THINK ABOUT IT

'First bad version' is a boundary in a sorted yes/no sequence, which is binary search. The parallel part is a different trade: instead of one probe per round, send several probes in the same round and keep the gap that contains the boundary. Fewer rounds, more total calls.

Basis: LeetCode 278, First Bad Version, plus a concurrent k-ary search follow-up.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    for v in range(1, n + 1):
        if is_bad_version(v):
            return v
```

TIME	SPACE
O(n)	**O(1)**
calls	

WHY IT IS SLOW

One call per version, O(n) calls; with a billion versions and a slow check that is far too many. The check is monotone, so binary search needs only log2(n) calls, and probing several points in parallel needs fewer rounds.

OPTIMAL SOLUTION · Base problem: binary search

TIME	SPACE
O(log n)	**O(1)**
calls	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Parallel: probe k points at once, then recurse into the bucket that flips

TIME	SPACE
O(log_{k+1} n) rounds	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What does parallelism buy you, and what does it cost?

It cuts the number of sequential rounds from log2(n) to log_{k+1}(n), so wall-clock time drops when each probe has high latency. It increases total probes (about k per round instead of 1), so it costs more calls and money. Say which metric you are optimizing before choosing k.

FOLLOW-UP 2 What if the oracle is rate limited or can fail?

Cap the pool size to the rate limit, retry transient failures with backoff (probes are idempotent), and never advance the interval from a probe that errored. Cache results by version so a retry or a later round never repeats a call.

FOLLOW-UP 3 Count how many rounds each approach needs.

Single binary search: log2(n) rounds of one probe each. k workers: about log base (k + 1) of n rounds with k probes each. For n = 10^9, one probe per round needs 30 rounds, while 7 probes per round need 10.

FOLLOW-UP 4 Show how many rounds and calls the parallel search uses.

With k workers each round shrinks the interval by about k + 1, so the number of sequential rounds is about log base (k + 1) of n and the total calls at most k times that. For n = 10^6 and 7 workers that is about 7 rounds and 49 calls, against 20 rounds and 20 calls for the sequential search: fewer rounds, more calls.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: First Bad Version with Parallel Search
# Follow-up
#
# Phone-screen R1 coding round. The base problem is the
# standard first-bad-version monotonic search — given an
# isBadVersion(v) oracle over versions 1..n, find the
# first bad version with a single binary search. The
# graded follow-up requires parallelizing the search:
# partition the range into buckets and search them
# concurrently (bucket + binary search), with working code
# expected.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What does parallelism buy you, and what does it cost?
# 2. What if the oracle is rate limited or can fail?
# 3. Count how many rounds each approach needs.
# 4. Show how many rounds and calls the parallel search
# uses.
#
# INTERVIEW NOTES
# 'First bad version' is a boundary in a sorted yes/no
# sequence, which is binary search. The parallel part is a
# different trade: instead of one probe per round, send
# several probes in the same round and keep the gap that
# contains the boundary. Fewer rounds, more total calls.
#
# BRUTE FORCE (say it first): Check versions one by one
# Call isBadVersion for 1, 2, 3, ... and stop at the first
# bad one. Obviously correct and the oracle; with a
# billion versions and a slow check it is far too many
# calls. Time O(n) calls. Space O(1).


from concurrent.futures import ThreadPoolExecutor
from threading import Lock


# BRUTE FORCE (say it first): Check versions one by one |
# time O(n) calls
def first_bad_linear(n, is_bad):
    return next(v for v in range(1, n + 1) if is_bad(v))


# BASE QUESTION (optimal): problem: binary search
# Idea: isBad is monotone (all good, then all bad), so
# keep the invariant 'the first bad version is in [lo,
# hi]' and halve the interval. Use lo + (hi - lo) // 2 in
# languages with fixed-width integers.
def first_bad_version(n, is_bad):
    # the base question: binary search on versions 1..n
    lo, hi = 1, n
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if is_bad(mid):
            # the first bad version is mid or earlier
            hi = mid
        else:
            lo = mid + 1
    return lo


# PARALLEL SEARCH helper
def probe_points(lo, hi, workers):
    # evenly spaced versions inside [lo, hi - 1]
    step = max(1, (hi - lo + 1) // (workers + 1))
    pts = {min(hi - 1, lo + step * (i + 1) - 1)
           for i in range(workers)}
    return sorted(pts)


# FOLLOW-UP 1: What does parallelism buy you, and what
# does it cost?
# Answer: It cuts the number of sequential rounds from
# log2(n) to log_{k+1}(n), so wall-clock time drops when
# each probe has high latency. It increases total probes
# (about k per round instead of 1), so it costs more calls
# and money.
# FOLLOW-UP 3: Count how many rounds each approach needs.
# Answer: Single binary search: log2(n) rounds of one
# probe each. k workers: about log base (k + 1) of n
# rounds with k probes each. For n = 10^9, one probe per
```

```
# round needs 30 rounds, while 7 probes per round need 10.
# FOLLOW-UP 4: Show how many rounds and calls the parallel
# search uses.
# Answer: With k workers each round shrinks the interval
# by about k + 1, so the number of sequential rounds is
# about log base (k + 1) of n and the total calls at most
# k times that.
def first_bad_parallel(n, is_bad, workers=4):
    # invariant: the answer is in [lo, hi]
```

```
    lo, hi, cache, rounds = 1, n, {}, 0
    with ThreadPoolExecutor(workers) as pool:
        while lo < hi:
            points = probe_points(lo, hi, workers)
            # ask the oracle about new points at the same
            # time
            fresh = [p for p in points if p not in cache]
            answers = pool.map(is_bad, fresh)
            for p, bad in zip(fresh, answers):
                cache[p] = bad
            rounds += 1
            bad_points = [p for p in points if cache[p]]
            if not bad_points:
                # every probe was good: answer is after
                # them
                lo = points[-1] + 1
            else:
                first = bad_points[0]
                # the answer sits between the last good
                # probe before it and the first bad one
                before = [p for p in points if p < first]
                if before:
                    lo = before[-1] + 1
                hi = first
    return lo, rounds, len(cache)


if __name__ == "__main__":
    calls, lock = [], Lock()

    def oracle(first_bad):
        def is_bad(v):
            with lock:
                calls.append(v)
            return v >= first_bad
        return is_bad

    cases = ((1, 1), (10, 1), (10, 10), (1000, 637))
    for n, bad in cases:
        assert first_bad_version(n, oracle(bad)) == bad
        ans, rounds, used = first_bad_parallel(
            n, oracle(bad), workers=4)
        assert ans == bad
    # fewer rounds than the 20 of a plain binary search
    ans, rounds, used = first_bad_parallel(
```

```
        10 ** 6, oracle(123456), workers=7)
    assert ans == 123456 and rounds < 20
    assert used <= 7 * rounds
```

NOTES FROM CANDIDATES

The base is trivial; budget your time for the parallel follow-up and aim to get a complete, compiling version through even if it has minor bugs — finishing the parallel version is what passes this round.

Be explicit about what the parallel design optimizes: wall-clock latency (fire many isBadVersion probes at once) versus total number of calls. Splitting into k buckets and probing each boundary concurrently localizes the transition in one parallel round, then a binary search finishes it.

Common bugs: off-by-one when mapping a bucket index back to an absolute version range; the transition sitting exactly on a bucket boundary; integer overflow in mid if you write (lo + hi) / 2 instead of lo + (hi - lo) // 2.

PREPARATION

Write the single-threaded binary search first and verify it on small n.

Then implement the bucketed parallel version with real concurrency primitives (Python concurrent.futures.ThreadPoolExecutor, or Java ExecutorService / CompletableFuture): split [1, n] into k buckets, probe the k boundaries concurrently, pick the bucket where the predicate flips, and binary-search within it. Practice getting this to compile and run in under 20 minutes.

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Binary search

MEDIUM | PracHub | Technical Screen | 4 of 12

★ TOP 75: rank 2 in the Medium and Hard priority list

★ Find median of merged RLE arrays

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 74.2 | 30 DAYS 61 | PRIORITY 5/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Binary search

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #4 Median of Two Sorted Arrays

LeetCode match: LC 4 Median of Two Sorted Arrays (RLE variant) (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/find-median-of-merged-rle-arrays

QUESTION

Overview: The question evaluates understanding of run-length encoding, order-statistics (median) computation, and efficient merging of frequency-annotated sequences, emphasizing handling large frequencies without explicit decompression.

APPROACH

Total length = sum of run counts; walk both run lists merged by value accumulating counts until the median index(es) are covered. Or binary search on value with counts.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether each array is sorted by value, how large counts can be (so decompression is impossible), and what to return for an even total (average of the two middle values).

BRUTE FORCE TO SAY OUT LOUD

Decompress both arrays, merge, take the middle. O(total count) memory, which counts can make huge.

OPTIMAL IDEA

Let N be the sum of all counts and the median positions be (N - 1) // 2 and N // 2. Merge the two run lists by value with two pointers (like the merge step of merge sort), accumulating counts until the running total passes each median position, and read the value there. O(r1 + r2) with no decompression. Alternative: binary search on the value, counting how many elements are at most that value in each array.

DRY-RUN THESE CASES

A single run, an even total, the median falling exactly on a run boundary, both arrays with the same values, an empty array.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

K arrays (heap merge), or arrays not sorted by value.

PITFALLS

Treating counts as lengths of different arrays instead of merging by value. Off-by-one at a run boundary.

HOW TO THINK ABOUT IT

Counts can be huge, so never expand a run. You only need the elements at ranks (N - 1) // 2 and N // 2 in the merged order. Merge the two run lists by value, counting how many elements you have passed, and read the value when the count reaches each rank.

Basis: Written; LeetCode 4 with run-length encoded inputs. Assumed input: two lists of (value, count) runs, each sorted by value; the median of the merged multiset (average of the two middle values for an even total).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(runs_a, runs_b):
    dense = []
    for value, count in runs_a + runs_b:
        dense += [value] * count
    dense.sort()
    n = len(dense)
    if n % 2:
        return dense[n // 2]
    mid = n // 2
    return (dense[mid - 1] + dense[mid]) / 2
```

TIME	SPACE
O(total count)	O(total count)

WHY IT IS SLOW

Needs time and memory proportional to the total count, which can be 10^9 or more. Walking the two run lists in value order and counting up to the middle rank costs only the number of runs.

OPTIMAL SOLUTION · Merge the runs and walk to the two middle ranks

TIME	SPACE
O(r1 + r2)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 K arrays instead of two.

Merge all runs by value (heap of K heads, or sort the combined run list if memory allows) and walk to the two middle ranks. O(R log K) with a heap, R the total number of runs.

FOLLOW-UP 2 The arrays are huge and sorted, and you want O(log) time.

Binary search on the value: for a candidate value count how many elements are at most it in each array (binary search on prefix sums of counts) and find the smallest value with rank above the middle. O(log(range) * log(runs)).

FOLLOW-UP 3 Runs are not sorted by value.

Sort the runs by value first, O(R log R), then use the same walk.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find median of merged RLE arrays
#
# The question evaluates understanding of run-length
# encoding, order-statistics (median) computation, and
# efficient merging of frequency-annotated sequences,
# emphasizing handling large frequencies without explicit
# decompression.
#
# FOLLOW-UPS (answered below, in this order):
# 1. K arrays instead of two.
# 2. The arrays are huge and sorted, and you want O(log)
# time.
# 3. Runs are not sorted by value.
#
# INTERVIEW NOTES
# Counts can be huge, so never expand a run. You only need
# the elements at ranks (N - 1) // 2 and N // 2 in the
# merged order. Merge the two run lists by value, counting
# how many elements you have passed, and read the value
# when the count reaches each rank.
#
# BRUTE FORCE (say it first): Expand both arrays, merge
# and take the middle
# Decompress each run into repeated values, merge, and
# read the middle. Correct, and the oracle; counts like
# 10^9 make the expansion impossible. Time O(total count).
# Space O(total count).


import heapq
from bisect import bisect_right
from itertools import accumulate


# BRUTE FORCE (say it first): Expand both arrays, merge
# and take the middle | time O(total count)
def median_rle_expand(a, b):
    data = sorted(
        [v for v, c in a for _ in range(c)]
        + [v for v, c in b for _ in range(c)]
    )
    n = len(data)
    return (
        data[n // 2]
        if n % 2
        else (data[n // 2 - 1] + data[n // 2]) / 2
    )
```

```
# HELPERS for the base question
def kth_smallest(runs_a, runs_b, k):
    # zero-based rank in the merged order, walking the
    # runs
    # of two sorted run-length lists; nothing is expanded
    i = j = seen = 0
    while True:
        take_a = j >= len(runs_b)
        if not take_a and i < len(runs_a):
            take_a = runs_a[i][0] <= runs_b[j][0]
        value, count = runs_a[i] if take_a else runs_b[j]
        seen += count
        if seen > k:
            return value
        if take_a:
            i += 1
        else:
            j += 1


# BASE QUESTION (optimal): Merge the runs and walk to the
# two middle ranks
# Idea: Let N be the sum of all counts.
def median_rle(a, b):
    # a and b: [(value, count)] sorted by value
    total = sum(c for _, c in a) + sum(c for _, c in b)
    if total % 2:
        return kth_smallest(a, b, total // 2)
    low = kth_smallest(a, b, total // 2 - 1)
    high = kth_smallest(a, b, total // 2)
    return (low + high) / 2


# FOLLOW-UP 1: K arrays instead of two.
# Answer: Merge all runs by value (heap of K heads, or
# sort the combined run list if memory allows) and walk to
# the two middle ranks. O(R log K) with a heap, R the
# total number of runs.
def median_k_arrays(arrays):
    # K arrays: merge the run lists by value, then walk to
    # the
    # middle ranks
    runs = list(heapq.merge(*arrays))
    total = sum(c for _, c in runs)

    def kth(k):
        seen = 0
        for value, count in runs:
            seen += count
            if seen > k:
                return value

    if total % 2:
        return kth(total // 2)
    return (kth(total // 2 - 1) + kth(total // 2)) / 2


# FOLLOW-UP 2: The arrays are huge and sorted, and you
# want O(log) time.
# Answer: Binary search on the value: for a candidate
# value count how many elements are at most it in each
# array (binary search on prefix sums of counts) and find
# the smallest value with rank above the middle.
# O(log(range) * log(runs)).
def median_by_value(arrays):
    # sorted arrays, whole-number values: binary search
    # the
    # value; count_at_most uses prefix sums of the counts
    prefixes = []
    for runs in arrays:
        values = [v for v, _ in runs]
```

```
        sums = list(accumulate(c for _, c in runs))
        prefixes.append((values, sums))

    def count_at_most(x):
        total = 0
        for values, sums in prefixes:
            i = bisect_right(values, x)
            total += sums[i - 1] if i else 0
        return total

    n = count_at_most(10 ** 18)

    def kth(k):
        lo, hi = -(10 ** 18), 10 ** 18
        while lo < hi:
```

```
            mid = (lo + hi) // 2
            # smallest value with more than k elements <=
            # it
            if count_at_most(mid) > k:
                hi = mid
            else:
                lo = mid + 1
        return lo

    if n % 2:
        return kth(n // 2)
    return (kth(n // 2 - 1) + kth(n // 2)) / 2


if __name__ == "__main__":
    a = [(1, 3), (4, 1)]
    b = [(2, 2), (5, 2)]
    # merged: 1 1 1 2 2 4 5 5 -> middle two are 2 and 2
    assert median_rle(a, b) == 2
    assert median_k_arrays([a, b]) == 2
    assert median_by_value([a, b]) == 2
    # an odd total: 1, 7, 7, 7, 7, 7 plus one more 7
    assert median_rle([(7, 6)], [(1, 1)]) == 7
    # huge counts are never expanded
    big = [(1, 10 ** 12)]
    assert median_rle(big, [(3, 10 ** 12)]) == 2
    c = [(0, 1), (10, 1)]
    three = [a, b, c]
    by_merge = median_k_arrays(three)
    assert by_merge == median_by_value(three)
```

FUNCTION AND TEST CASES

Sample call

```
solution([(1, 2), (5, 1)], [(2, 3)])
```

Test cases

Input	Expected
`([(1, 2), (5, 1)], [(2, 3)])`	`2.0`
`([(1, 1)], [(2, 1)])`	`1.5`

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Binary search

MEDIUM | PracHub | Technical Screen | 5 of 12

★ TOP 75: rank 12 in the Medium and Hard priority list

★ Solve Banana Speed and Interval Merge

POSTED 2025-01-27 | 631 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 65.2 | 30 DAYS 41.1 | PRIORITY 10/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Binary search

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #875 Koko Eating Bananas, LeetCode #56 Merge Intervals

LeetCode match: LC 875 Koko Eating Bananas + LC 56 Merge Intervals

Posted on PracHub: 2025-01-27 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/solve-banana-speed-and-interval-merge

QUESTION

During two technical phone screens for a new graduate software engineering role, the candidate was asked to solve two coding problems: minimum eating speed and merging overlapping intervals.

Constraints & Assumptions

- Provide clear algorithms, correctness reasoning, complexity, and edge cases.
- For minimum eating speed, each hour can be spent on only one pile.
- For interval merging, intervals overlap when the next start is less than or equal to the current merged end.
- Use sorted or binary-search approaches where appropriate.

Clarifying Questions

to Ask - Are **`piles`** and **h** always positive?

- Is **h** at least the number of piles?
- Are interval endpoints inclusive?
- Can intervals be empty or unsorted?

Part 1

- Minimum Eating Speed Given **`piles`** and **h**, return the minimum integer speed **k** such that all piles can be eaten within **h** hours.

What This Part Should Cover

- Binary search on answer from 1 to **`max(piles)`**.
- Feasibility function using sum of **`ceil(pile / k)`**.
- Why feasibility is monotonic.
- Complexity and edge cases.

Part 2

QUESTION

- Merge Overlapping Intervals Given intervals **`[start, end]`**, merge all overlapping intervals and return non-overlapping intervals sorted by start.

What This Part Should Cover

- Sort by start.
- Maintain current merged interval.
- Extend on overlap, otherwise append and start a new interval.
- Complexity and edge cases.

What a Strong

Answer Covers - Recognizes monotonic binary search for eating speed.

- Uses integer-safe ceiling division.
- Correctly defines interval overlap.
- Gives clean code and complexity for both problems.

Follow-up Questions

- How would you avoid floating-point division in the banana problem?
- What if **h** is smaller than the number of piles?
- How do you merge touching intervals like **`[1,3]`** and **`[3,5]`**?
- How would you handle streaming intervals?Minimum Eating Speed (Koko Eating Bananas)Koko loves to eat bananas. There are **n** piles of bananas; the i-th pile has **`piles[i]`** bananas. The guards will return in **h** hours. Koko can decide her constant bananas-per-hour eating speed **k**. Each hour she chooses one pile and eats **k** bananas from it. If that pile has fewer than **k** bananas, she eats all of them and does not eat more during that hour. An hour is spent on only one pile. Return the minimum integer speed **k** such that she can finish all the bananas within **h** hours. Example:
- **`piles = [3, 6, 7, 11]`**, **`h = 8`** -> **`4`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 5`** -> **`30`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 6`** -> **`23`**
  Constraints: **`1 <= piles.length <= h`**, **`1 <= piles[i] <= 10^9`**.

Constraints

1 <= piles.length <= h1 <= piles[i] <= 10^9h can be as large as 10^9

EXAMPLES

```
Input: ([3, 6, 7, 11], 8)
Expected Output: 4
Explanation: At k=4:
    ceil(3/4)+ceil(6/4)+ceil(7/4)+ceil(11/4) =
    1+2+2+3 = 8 hours. k=3 needs 1+2+3+4 = 10 > 8, so
    4 is minimal.
Input: ([30, 11, 23, 4, 20], 5)
Expected Output: 30
```

Explanation: With exactly 5 piles and 5 hours, each pile must be eaten in one hour, so k must be at least max(piles) = 30.

QUESTION

Hints

If speed k finishes in time, any larger speed also finishes in time. This monotonicity lets you binary search on the answer.The answer lies in [1, max(piles)]: speed 1 is the slowest, and max(piles) finishes every pile in at most one hour.Hours needed at speed k is sum of ceil(pile / k). Use integer-safe ceiling: (pile + k - 1) // k to avoid floating point.Merge Overlapping IntervalsGiven an array of intervals where **`intervals[i] = [start_i, end_i]`**, merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input, sorted by start. Two intervals overlap when the next start is less than or equal to the current merged end

APPROACH

Binary search the speed with ceil(pile/speed) sum <= h; sort intervals by start and merge overlaps in one pass.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Bananas: piles and h are positive, h is at least the number of piles (otherwise impossible), one pile per hour, and an hour finishes at most one pile. Intervals: overlap includes touching (next start at most the current end).

BRUTE FORCE TO SAY OUT LOUD

Bananas: try every speed from 1 up. Intervals: compare every pair.

OPTIMAL IDEA

Bananas (LeetCode 875): the time function is monotone in the speed, so binary search the smallest speed in [1, max(piles)] with sum(ceil(p / s)) <= h. O(n log max). Intervals: sort by start and sweep, merging while the next start is at most the current end. O(n log n).

DRY-RUN THESE CASES

Bananas: h equal to the pile count (speed is the maximum), a single pile, large piles (overflow). Intervals: touching, nested, unsorted, one interval.

LIKELY FOLLOW-UPS

Bananas with a variable h or a floating-point speed. Intervals: insert one interval or count the merged result.

PITFALLS

Using floor instead of ceil. Merging with strict less-than.

HOW TO THINK ABOUT IT

Bananas: the time needed is non-increasing in the speed, so the smallest speed that finishes in h hours can be found by binary search over speeds, with a simple sum of ceil(pile / speed) as the check. Intervals: sort by start and sweep with one 'current' interval; the sort guarantees nothing earlier can still overlap.

HOW TO THINK ABOUT IT

Basis: LeetCode 875 (Koko Eating Bananas; one pile per hour, h at least the number of piles) and LeetCode 56 (Merge Intervals; overlapping when the next start is at most the current end).

BRUTE FORCE · Bananas: try every speed upward

BRUTE FORCE · PSEUDO-CODE

```
def brute(piles, h):
    for speed in range(1, max(piles) + 1):
        hours = 0
        for p in piles:
            hours += ceil(p / speed)
        if hours <= h:
            return speed
```

TIME	SPACE
O(max * n)	O(1)

WHY IT IS SLOW

Up to max(piles) speeds, each costing O(n); piles can be 10^9. Hours are non-increasing in the speed, so binary search over speeds is O(n log max).

BRUTE FORCE · Intervals: merge any overlapping pair until none is left

BRUTE FORCE · PSEUDO-CODE

```
def merge(intervals):
    intervals = list(intervals)
    merged = True
    while merged:
        merged = False
        for a, b in combinations(intervals, 2):
            if overlaps(a, b):
                intervals.remove(a)
                intervals.remove(b)
                intervals.append(union(a, b))
                merged = True
                break
    return sorted(intervals)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION · Bananas: binary search the speed

TIME	SPACE
O(n log max)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Intervals: sort and sweep

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Bananas: h varies per query, or the piles change.

The check is independent of the search; for many queries keep the piles sorted with prefix sums so the hours needed at speed s can be computed in O(log n) per distinct pile size.

FOLLOW-UP 2 Intervals: insert one new interval into an already merged list (LeetCode 57).

Copy intervals ending before the new one, merge the ones that overlap it, then copy the rest. O(n), no sort needed.

FOLLOW-UP 3 Bananas: the guards come back in fewer hours than piles.

Then no speed works (an hour can finish at most one pile): check h >= len(piles) first and return -1 otherwise.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve Banana Speed and Interval Merge
#
# During two technical phone screens for a new graduate
# software engineering role, the candidate was asked to
# solve two coding problems: minimum eating speed and
# merging overlapping intervals. ### Constraints &
# Assumptions - Provide clear algorithms, correctness
# reasoning, complexity, and edge cases. - For minimum
# eating speed, each hour can be spent on only one pile. -
# For interval merging, intervals overlap when the next
# start is less than or equal to the current merged end. -
# Use sorted or binary-search approaches where
# appropriate. ### Clarifying Questions to Ask - Are piles
# and h always positive? - Is h at least the number of
# piles? - Are interval endpoints inclusive? - Can
# intervals be empty or unsorted? ### Part 1 - Minimum
# Eating Speed Given piles and h, return the minimum
# integer speed k such that all piles can be eaten within
# h hours.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Bananas: h varies per query, or the piles change.
# 2. Intervals: insert one new interval into an already
# merged list (LeetCode 57).
# 3. Bananas: the guards come back in fewer hours than
# piles.
#
# INTERVIEW NOTES
# Bananas: the time needed is non-increasing in the speed,
# so the smallest speed that finishes in h hours can be
# found by binary search over speeds, with a simple sum of
# ceil(pile / speed) as the check. Intervals: sort by
# start and sweep with one 'current' interval; the sort
# guarantees nothing earlier can still overlap.
#
# BRUTE FORCE (say it first): Bananas: try every speed
# upward
# For speed 1, 2, 3, ... compute the hours needed and
# return the first speed that fits in h. Correct and the
# oracle; the speed can be 10^9. Time O(max * n). Space
# O(1).


# BRUTE FORCE (say it first): Bananas: try every speed
# upward | time O(max * n)
```

```
def banana_speed_linear(piles, h):
    for speed in range(1, max(piles) + 1):
        if sum(-(-p // speed) for p in piles) <= h:
            return speed


# HELPERS for the base question
def hours_needed(piles, speed):
    # each hour she works on ONE pile, so every pile
    # rounds up
    return sum(-(-p // speed) for p in piles)


# BASE QUESTION (optimal): Bananas: binary search the
# speed
# Idea: Hours needed are non-increasing in the speed, so
# keep lo = 1 and hi = max(piles) and halve: if speed mid
# finishes within h, try slower, otherwise faster. Use
# ceiling division (an hour finishes at most one pile).
def min_eating_speed(piles, h):
    # fewer hours than piles can never work
    if h < len(piles):
        return -1
    lo, hi = 1, max(piles)
    # more speed never takes more hours: binary search
    while lo < hi:
        mid = (lo + hi) // 2
        if hours_needed(piles, mid) <= h:
            hi = mid
        else:
            lo = mid + 1
    return lo


# PART 2: merge intervals
def merge_intervals(intervals):
    out = []
    for s, e in sorted(intervals):
        # overlap (sharing a point counts): extend the
        # last one
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

```
# FOLLOW-UP 2: Intervals: insert one new interval into an
# already merged list (LeetCode 57).
# Answer: Copy intervals ending before the new one, merge
# the ones that overlap it, then copy the rest. O(n), no
# sort needed.
def insert_interval(merged, new):
    # LeetCode 57: the list is already merged and sorted:
    # O(n)
    out, i, n = [], 0, len(merged)
    # 1) intervals that end before the new one starts
```

```
    while i < n and merged[i][1] < new[0]:
        out.append(merged[i])
        i += 1
    # 2) intervals that overlap it are absorbed
    s, e = new
    while i < n and merged[i][0] <= e:
        s, e = min(s, merged[i][0]), max(e, merged[i][1])
        i += 1
    out.append([s, e])
    # 3) the rest comes after
    out += merged[i:]
    return out


if __name__ == "__main__":
    assert min_eating_speed([3, 6, 7, 11], 8) == 4
    assert min_eating_speed([30, 11, 23, 4, 20], 5) == 30
    assert min_eating_speed([30, 11, 23, 4, 20], 6) == 23
    # fewer hours than piles: impossible
    assert min_eating_speed([1, 1, 1], 2) == -1
    assert hours_needed([3, 6, 7, 11], 4) == 8
    spans = [[1, 3], [2, 6], [8, 10], [15, 18]]
    merged = merge_intervals(spans)
    assert merged == [[1, 6], [8, 10], [15, 18]]
    assert merge_intervals([[1, 4], [4, 5]]) == [[1, 5]]
    assert merge_intervals([]) == []
    got = insert_interval([[1, 3], [6, 9]], [2, 5])
    assert got == [[1, 5], [6, 9]]
    assert insert_interval([], [4, 8]) == [[4, 8]]
    assert insert_interval([[1, 2], [5, 6]], [3, 4]) == [
        [1, 2], [3, 4], [5, 6]]
```

FUNCTION AND TEST CASES

Signature

```
def solution(piles: list[int], h: int) -> int:
```

Sample call

```
solution([3, 6, 7, 11], 8)
```

Test cases

Input	Expected
([3, 6, 7, 11], 8)	4
([30, 11, 23, 4, 20], 5)	30

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Binary search

HARD | IN APP LC 354 | PracHub | Technical Screen | 6 of 12

★ TOP 75: rank 14 in the Medium and Hard priority list

★ Find Maximum Envelope Nesting

POSTED 2026-03-18 | 216 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 52.8 | 30 DAYS 33 | PRIORITY 27/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Binary search

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #354 Russian Doll Envelopes

LeetCode match: LC 354 Russian Doll Envelopes

Posted on PracHub: 2026-03-18 (updated 2026-05-06)

Original: https://prachub.com/coding-questions/find-maximum-envelope-nesting

QUESTION

Overview: This question evaluates algorithmic problem-solving skills, particularly reasoning about multi-dimensional ordering, sequence optimization, and time-complexity analysis.

APPROACH

Sort by width asc and height desc on ties, then find the strictly increasing subsequence on heights with binary-search LIS (patience sorting). O(n log n).

HOW TO THINK ABOUT IT

Two dimensions are hard, one is easy. Sort by width so that width order is settled, then the question becomes a longest strictly increasing subsequence of heights. The trap is envelopes with equal width: they cannot nest, so sort equal widths by height descending, which prevents two of them from forming an increasing pair. The LIS is then done with patience sorting (binary search over tails).

Basis: LeetCode 354, Russian Doll Envelopes (strictly larger in both width and height).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
envelopes.sort()  # by width
best = [1] * len(envelopes)
for i in range(len(envelopes)):
    for j in range(i):
        w1, h1 = envelopes[j]
        w2, h2 = envelopes[i]
        if w1 < w2 and h1 < h2:
            best[i] = max(best[i], best[j] + 1)
answer = max(best)
```

BRUTE FORCE

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every envelope compares with all earlier ones, O(n^2). Sorting by width ascending and height descending, then a longest increasing subsequence on heights with binary search, is O(n log n).

OPTIMAL SOLUTION · Sort by width ascending and height descending, then LIS by binary search

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the actual chain of envelopes.

Remember, for each element, the index of the tail it extended (parent pointers), and rebuild from the last tail of the longest length. The sorted order and binary search are unchanged.

FOLLOW-UP 2 Three dimensions (boxes).

Sorting plus LIS no longer reduces the problem; use an O(n^2) DP, or CDQ divide and conquer with a Fenwick tree for O(n log^2 n).

FOLLOW-UP 3 Non-strict nesting (equal sizes allowed).

Sort by width ascending and height ascending, and use bisect_right (longest non-decreasing subsequence) instead of bisect_left.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find Maximum Envelope Nesting
#
# This question evaluates algorithmic problem-solving
# skills, particularly reasoning about multi-dimensional
# ordering, sequence optimization, and time-complexity
# analysis.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the actual chain of envelopes.
# 2. Three dimensions (boxes).
# 3. Non-strict nesting (equal sizes allowed).
#
# INTERVIEW NOTES
# Two dimensions are hard, one is easy. Sort by width so
# that width order is settled, then the question becomes a
# longest strictly increasing subsequence of heights. The
# trap is envelopes with equal width: they cannot nest, so
```

```python
# sort equal widths by height descending, which prevents
# two of them from forming an increasing pair. The LIS is
# then done with patience sorting (binary search over
# tails).
#
# BRUTE FORCE (say it first): DP over pairs: longest chain
# ending at each envelope
# Sort by width, then best[i] is 1 plus the best best[j]
# over earlier envelopes that are strictly smaller in both
# dimensions. Correct and the oracle; quadratic. Time
# O(n^2). Space O(n).


from bisect import bisect_left, bisect_right


# BRUTE FORCE (say it first): DP over pairs: longest chain
# ending at each envelope | time O(n^2)
def nesting_dp(envelopes):
    env = sorted(envelopes)
    best = [1] * len(env)
    for i in range(len(env)):
        for j in range(i):
            if (
                env[j][0] < env[i][0]
                and env[j][1] < env[i][1]
            ):
                best[i] = max(best[i], best[j] + 1)
```

```python
    return max(best, default=0)


# BASE QUESTION (optimal): Sort by width ascending and
# height descending, then LIS by binary search
# Idea: Sorting equal widths by decreasing height means
# two envelopes of the same width can never both appear in
# a strictly increasing height sequence.
def max_envelopes(envelopes):
    # sort by width, taller first on ties, then find the
    # longest
    # strictly increasing run of heights
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    tails = []
    for _, h in ordered:
        i = bisect_left(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)


# FOLLOW-UP 1: Return the actual chain of envelopes.
# Answer: Remember, for each element, the index of the
# tail it extended (parent pointers), and rebuild from the
# last tail of the longest length. The sorted order and
# binary search are unchanged.
def envelope_chain(envelopes):
    # also return the chain: remember what each element
    # extended
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    tails, tail_idx = [], []
    prev = [-1] * len(ordered)
    for i, (_, h) in enumerate(ordered):
        k = bisect_left(tails, h)
        if k == len(tails):
            tails.append(h)
            tail_idx.append(i)
        else:
            tails[k] = h
            tail_idx[k] = i
```

```python
            # the element that ended a chain one shorter
```

```python
            prev[i] = tail_idx[k - 1] if k else -1
    chain = []
    i = tail_idx[-1] if tail_idx else -1
    while i != -1:
        chain.append(ordered[i])
        i = prev[i]
    return chain[::-1]


# FOLLOW-UP 2: Three dimensions (boxes).
# Answer: Sorting plus LIS no longer reduces the problem;
# use an O(n^2) DP, or CDQ divide and conquer with a
# Fenwick tree for O(n log^2 n).
def max_boxes_3d(boxes):
    # three sizes: the reduction no longer works, use
    # O(n^2)
    ordered = sorted(boxes)
    best = [1] * len(ordered)
    for i in range(len(ordered)):
        for j in range(i):
            a, b = ordered[j], ordered[i]
            if all(x < y for x, y in zip(a, b)):
                best[i] = max(best[i], best[j] + 1)
    return max(best, default=0)


# FOLLOW-UP 3: Non-strict nesting (equal sizes allowed).
# Answer: Sort by width ascending and height ascending,
# and use bisect_right (longest non-decreasing
# subsequence) instead of bisect_left.
def max_nested_non_strict(envelopes):
    # equal sizes may nest: sort both ways ascending and
    # use
    # bisect_right (a longest non-decreasing subsequence)
    tails = []
    for _, h in sorted(envelopes):
        i = bisect_right(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)


if __name__ == "__main__":
```

```python
    envs = [[5, 4], [6, 4], [6, 7], [2, 3]]
    assert max_envelopes(envs) == 3
    chain = envelope_chain(envs)
    assert len(chain) == 3
    # every envelope fits strictly inside the next
    for a, b in zip(chain, chain[1:]):
        assert a[0] < b[0] and a[1] < b[1]
    assert envelope_chain([]) == []
    # equal sizes: only the non-strict version nests them
    twins = [[3, 3], [3, 3], [4, 4]]
    assert max_envelopes(twins) == 2
    assert max_nested_non_strict(twins) == 3
    boxes = [(1, 1, 1), (2, 2, 2), (3, 1, 4), (3, 3, 3)]
    assert max_boxes_3d(boxes) == 3
```

FUNCTION AND TEST CASES

Signature

```
def solution(envelopes: list[list[int]]) -> int:
SAMPLE INPUT
[[5, 4], [6, 4], [6, 7], [2, 3]]
```

Test cases

Input	Expected
`[[5, 4], [6, 4], [6, 7], [2, 3]]`	`3`
`[]`	`0`

ALSO REPORTED · 1 more

Russian Doll Envelopes | 1Point3Acres | phone-screen | Single report

https://www.1point3acres.com/interview/problems/company/google/russian-doll-envelopes

Sorting heights descending on width-ties prevents same-width envelopes from chaining — easy to miss.
LIS via bisect_left on a tails array is the standard O(n log n) implementation.

Preparation: Write LIS O(n log n) from memory in under 8 min.
Justify the descending-height-on-tie trick out loud.
Plain LIS and the mountain-LIS variant make good drill partners.

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Sorting

Site tags: array, hashmap, sorting, medium

MEDIUM | 1Point3Acres | phone-screen | 7 of 12

Progressive Array Partitioning

ASKED 2026-08-11 | 70 DAYS AGO | LAST 90 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 198/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Sorting

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-11 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/array-partition-unique-frequencies-straights

QUESTION

EXAMPLES

Stage 1

```
[1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true because the
    four value-groups have different sizes.
[1, 2, 3] -> false because every value-group has size
    one.
```

Stage 2

```
[1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true.
[4, 5, 6, 7, 7, 8, 9] -> false.
```

APPROACH

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the groups are and the rule for each stage. From the examples, the first stage looks like: true when the counts of all distinct values are different from each other (1:1, 2:2, 3:3, 4:4 is true; all counts 1 is false). The second stage is not clear from the examples alone, so ask for its exact rule and test it on both examples before coding.

BRUTE FORCE TO SAY OUT LOUD

Try every way to group the elements and test the rule. Exponential.

OPTIMAL IDEA

For stage one: count each value with a hash map, then check that the multiset of counts has no repeats (a set of counts has the same size as the number of values). O(n). Keep the data model (a counter) easy to adapt, because the second stage arrives only after the first is written.

DRY-RUN THESE CASES

The four examples in the statement, an empty array, a single value, many values with equal counts.

LIKELY FOLLOW-UPS

Stage two, which changes the grouping rule; keep helper functions small so it is a local change.

PITFALLS

Coding stage two from a guess. Not reconciling the rule with both examples.

HOW TO THINK ABOUT IT

The statement arrives in stages, so keep the data model easy to adapt: count the values first, because any rule about 'groups of equal values' is a rule about the counts. Stage 1 is then 'are all counts different', which is a set-size comparison. Before stage 2, reconcile the new rule with every example; here a natural guess (LeetCode 659) contradicts the second example, which is exactly the situation where you ask.

Basis: Written. Stage 1 is inferred from the examples: true when every distinct value occurs a different number of times. The Stage 2 rule is NOT recoverable from the two examples, so only a candidate reading is shown; ask the interviewer for the exact rule and test it against both examples before coding.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    counts = Counter(a)
    for x, y in combinations(counts, 2):
        if counts[x] == counts[y]:
            return False
    return True
```

TIME	SPACE
O(d^2) where d is the number of distinct values	O(d)

WHY IT IS SLOW

O(d^2) pair checks for d distinct values. Comparing the number of distinct counts with the number of distinct values (a set) is O(n).

OPTIMAL SOLUTION · Counter, then compare the set of counts with the number of values

TIME	SPACE
O(n)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Stage 2 arrives. What do you do?

Do not guess. Write down what the second example pair implies, and check any candidate rule against BOTH. The common reading 'split into consecutive runs of length at least 3' gives true for [4, 5, 6, 7, 7, 8, 9], but the statement says false, so there is an extra condition (such as run lengths or a count rule) to ask about.

FOLLOW-UP 2 Keep the code adaptable between stages.

Put the counting in one helper and each stage's rule in its own small function that takes the counts. Stage 2 then replaces a function instead of rewriting the pipeline.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Progressive Array Partitioning
#
# Examples: Stage 1 [1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true
# because the four value-groups have different sizes. [1,
# 2, 3] -> false because every value-group has size one.
# Stage 2 [1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true. [4, 5, 6
,
# 7, 7, 8, 9] -> false.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Stage 2 arrives. What do you do?
# 2. Keep the code adaptable between stages.
#
# INTERVIEW NOTES
# The statement arrives in stages, so keep the data model
# easy to adapt: count the values first, because any rule
# about 'groups of equal values' is a rule about the
# counts. Stage 1 is then 'are all counts different',
# which is a set-size comparison. Before stage 2,
# reconcile the new rule with every example; here a
# natural guess (LeetCode 659) contradicts the second
# example, which is exactly the situation where you ask.
#
# BRUTE FORCE (say it first): Compare every pair of counts
# Count each value, then check every pair of counts for
# equality. Fine for small d and a clear statement of the
# rule. Time O(d^2) where d is the number of distinct
# values. Space O(d).


from collections import Counter


# BRUTE FORCE (say it first): Compare every pair of counts
# | time O(d^2) where d is the number of distinct values
def distinct_counts_brute(nums):
    counts = list(Counter(nums).values())
    return all(
        counts[i] != counts[j]
        for i in range(len(counts))
        for j in range(i + 1, len(counts))
    )


# BASE QUESTION (optimal): Counter, then compare the set
# of counts with the number of values
```

```
# Idea: All counts are different exactly when the set of
# counts has as many elements as there are distinct
# values. One pass to count, one pass over the counts.
def stage1(counts):
    # true when every value occurs a different number of
    # times: the counts have no repeats
    return len(set(counts.values())) == len(counts)


# FOLLOW-UP 1: Stage 2 arrives. What do you do?
# Answer: Do not guess. Write down what the second example
# pair implies, and check any candidate rule against BOTH.
def stage2_candidate(nums, min_len=3):
    # ONE reading of stage 2 (LeetCode 659): split the
    # array into runs of consecutive values, each at
    # least min_len long. Check it against BOTH examples.
    left, ends = Counter(nums), Counter()
    for x in nums:
        if left[x] == 0:
            continue
        left[x] -= 1
        if ends[x - 1] > 0:
            # extend a run that ended at x - 1
            ends[x - 1] -= 1
            ends[x] += 1
        elif left[x + 1] > 0 and left[x + 2] > 0:
            # start a new run x, x + 1, x + 2
            left[x + 1] -= 1
            left[x + 2] -= 1
            ends[x + 2] += 1
        else:
            return False
    return True


# HELPERS for the checks below
def counts_of(nums):
    # every rule about groups of equal values is a
    # rule about these counts, so compute them once
    return Counter(nums)


# each stage is one small function, so a new rule
# replaces one entry instead of the whole pipeline
RULES = {1: lambda nums: stage1(counts_of(nums)),
```

```
         2: stage2_candidate}


def check(stage, nums):
    return RULES[stage](nums)


if __name__ == "__main__":
    # stage 1 examples
    assert check(1, [1, 2, 3, 4, 2, 3, 3, 4, 4, 4])
    assert not check(1, [1, 2, 3])
    # stage 2 example that the candidate gets right
    assert check(2, [1, 2, 2, 3, 3, 4, 4, 5, 5, 6])
    # the second stage-2 example says False, but this
    # reading says True: the rule is incomplete, so ask
    assert check(2, [4, 5, 6, 7, 7, 8, 9]) is True
```

NOTES FROM CANDIDATES

The interviewer presents Stage 2 only after the first implementation, so keep the initial data model easy to adapt.
Before coding the straight variant, reconcile the length rule with both examples; otherwise a superficially valid decomposition can contradict the expected result.
Evaluation emphasized the efficiency of the chosen data structures, not only whether the sample cases passed.
Solution skeleton
Stage 1: count each distinct value, then verify that the frequency values are themselves unique. A hash map plus a set gives O(n) time and O(u) space for u distinct values.
Stage 2: first pin down a group size k. For the fixed-size interpretation, reject when n % k != 0, process keys in ascending order, and whenever the smallest remaining key x has multiplicity c, remove c copies from every key in [x, x + k). Any missing count makes the partition impossible. Sorting the u keys gives O(n + u log u) time and O(u) space.
Do not let length-one groups make every input trivially valid unless the interviewer explicitly allows them; the negative example requires a stronger group-size contract.

PREPARATION

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Sorting

Site tags: data-structure, sorting, sliding-window, medium, ambiguous-spec

MEDIUM | 1Point3Acres | onsite-coding | 8 of 12

Streaming Points: Emit K Within Distance

ASKED 2026-06-23 | 119 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 199/228**

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Sorting

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-points-k-within-distance

QUESTION

Process a stream of float positions. Whenever three points satisfy the distance condition, output and remove those three points; a follow-up generalizes from 3 points to K points. The important ambiguity is whether the condition means a span of length **d** or pairwise distance within **d**.SWEdata-structuresorting sliding-windowmediumambiguous-specFrequencySingle reportLast asked2026-06-23Stageonsite-coding

APPROACH

Practice range queries over sorted points with insertion and deletion.
Write the K-generalization in words before coding: identify the local candidate window, test the predicate, then remove exactly the emitted points.
Rehearse explaining float comparison, tie-breaking, and what to do when multiple groups qualify.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the condition means all K points fit in a span of length d or every pair is within d (on a line these are equivalent), whether to emit as soon as a group exists, and how to choose among overlapping groups.

BRUTE FORCE TO SAY OUT LOUD

Keep a list, and after each point sort it and scan for a group. O(n log n) per point.

OPTIMAL IDEA

Keep points in a sorted list (bisect.insort). After inserting x only windows containing x can be new, so look at the K-wide windows of sorted neighbors around its index; if the largest minus the smallest in a window is at most d, emit and remove those K points. O(log n) to locate plus the window size.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

Exactly d apart, duplicates, points arriving out of order, two overlapping candidate groups, K = 1.

LIKELY FOLLOW-UPS

General K (the same code), 2D points, or a time-based expiration.

PITFALLS

Using epsilon-free float comparison blindly. Checking only the K points directly before x.

HOW TO THINK ABOUT IT

On a line, K points are all within distance d exactly when the largest minus the smallest is at most d, so only K consecutive points in sorted order can form a group. Keep the points sorted and look only where something changed: a window that contains the new point, or the seam left by removed points.

Basis: Written. Assumed rules (confirm them): after each new point, while some K consecutive points (in sorted order) span at most d, emit the lowest such group and remove it; on a line 'all within d' and 'span at most d' are the same condition.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
points = []
def on_point(x):
    points.append(x)
    points.sort()
    emitted = []
    while True:
        group = find_group()
        if group is None:
            return emitted
        emitted.append(group)
        remove(points, group)
def find_group():
    for i in range(len(points) - k + 1):
        if points[i + k - 1] - points[i] <= d:
            return points[i:i + k]
    return None
```

TIME	SPACE
O(n log n) per point	**O(n)**

WHY IT IS SLOW

Every arrival re-sorts and rescans the whole list, O(n log n). A sorted list with binary search checks only the windows around the new point.

OPTIMAL SOLUTION · Sorted list, check only windows touching the change

TIME	SPACE
O(log n + K) per check, plus list shifting	**O(n)**

OPTIMAL SOLUTION · Sorted list, check only windows touching the change

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if 'within d' means every pair, and points are in 2D?

On a line pairwise distance equals the span, so nothing changes. In 2D it becomes a clique problem: candidates must be neighbours in a spatial grid of cell size d, and you test the K-subset of nearby points; that is much harder, so ask whether the 1D reading is intended.

FOLLOW-UP 2 Floating-point coordinates.

Compare with the exact subtraction hi - lo <= d (not by building a range around the target), and agree on an epsilon policy only if the interviewer wants one. Do not compare floats for equality.

FOLLOW-UP 3 Several groups qualify at once.

Agree on a rule (lowest start, as coded) so output is deterministic, and note that emitting one group can create or destroy others.

FOLLOW-UP 4 The condition is 'every pair within d', not 'span at most d'.

On a line they are the same: the largest pairwise distance is max - min, so checking the span is enough (verified against checking all pairs). In 2D they differ and become a clique problem.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Streaming Points: Emit K Within Distance
#
# Process a stream of float positions. Whenever three
# points satisfy the distance condition, output and remove
# those three points; a follow-up generalizes from 3
# points to K points. The important ambiguity is whether
# the condition means a span of length d or pairwise
# distance within d.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if 'within d' means every pair, and points are
# in 2D?
# 2. Floating-point coordinates.
# 3. Several groups qualify at once.
# 4. The condition is 'every pair within d', not 'span at
# most d'.
#
# INTERVIEW NOTES
# On a line, K points are all within distance d exactly
# when the largest minus the smallest is at most d, so
# only K consecutive points in sorted order can form a
# group. Keep the points sorted and look only where
# something changed: a window that contains the new point,
# or the seam left by removed points.
#
# BRUTE FORCE (say it first): Re-sort and scan every
# window after each point
# Sort all points, slide a window of K, and emit the
# lowest group that fits, repeating until none does.
# Obviously correct; the whole list is re-examined for
# every new point. Time O(n log n) per point. Space O(n).


from bisect import bisect_left, insort


# BRUTE FORCE (say it first): Re-sort and scan every
# window after each point | time O(n log n) per point
class PointStreamBrute:
    def __init__(self, k, d):
        self.k, self.d, self.pts = k, d, []

    def add(self, x):
        self.pts.append(x)
        out = []
        while True:
            self.pts.sort()
            for i in range(len(self.pts) - self.k + 1):
                if (
                    self.pts[i + self.k - 1]
                    - self.pts[i]
                    <= self.d
                ):
                    out.append(self.pts[i : i + self.k])
                    del self.pts[i : i + self.k]
                    break
            else:
                return out


# BASE QUESTION (optimal): Sorted list, check only windows
# touching the change
# Idea: Insert with bisect. Because no window existed
# before, a new window must contain the new point, so test
# the K windows that include its position.
class PointStream:
    def __init__(self, k, d):
        # emit when k points fit in a span of d
        self.k, self.d = k, d
        # all points still waiting, kept sorted
        self.points = []

    def window_start(self, pos):
        # first window of k neighbours that includes pos
        lo = max(0, pos - self.k + 1)
        hi = min(pos, len(self.points) - self.k)
        for i in range(lo, hi + 1):
            last = self.points[i + self.k - 1]
            span = last - self.points[i]
            if span <= self.d:
                return i
        return None

    def add(self, x):
        # insert in order, then emit every window made by
        # x
        insort(self.points, x)
        out, pos = [], bisect_left(self.points, x)
        while True:
            i = self.window_start(pos)
```

```
            if i is None:
                return out
            # remove the group so it is emitted only once
            out.append(self.points[i:i + self.k])
            del self.points[i:i + self.k]
            pos = i


# FOLLOW-UP 4: The condition is 'every pair within d', not
# 'span at most d'.
# Answer: On a line they are the same: the largest
# pairwise distance is max - min, so checking the span is
# enough (verified against checking all pairs). In 2D they
# differ and become a clique problem.
def all_pairs_within(group, d):
    # "every pair within d": on a line this equals the
    # span
    pairs = [(a, b) for a in group for b in group]
    return all(abs(a - b) <= d for a, b in pairs)


if __name__ == "__main__":
    s = PointStream(3, 1.0)
    assert s.add(0.5) == []
    assert s.add(5.0) == []
    assert s.add(1.0) == []
    # 0.5, 1.0, 1.4 span 0.9 <= 1.0
    out = s.add(1.4)
    assert out == [[0.5, 1.0, 1.4]]
    # the far point is still waiting
    assert s.points == [5.0]
    # on a line, span and pairwise checks agree
    for group in out:
        assert all_pairs_within(group, 1.0)
        assert max(group) - min(group) <= 1.0
    # k = 2 groups closest neighbours as they appear
    t = PointStream(2, 0.5)
    assert t.add(1.0) == []
    assert t.add(1.4) == [[1.0, 1.4]]
```

NOTES FROM CANDIDATES

A balanced ordered set / TreeSet-style structure is a natural fit because each new point only needs nearby neighbors.
Be careful with floating-point comparison and epsilon policy.
A wrong clarification can lead to checking subset(target, target + d) when the interviewer expects a wider candidate range around the target. Confirm the predicate with a tiny numeric example before coding.

PREPARATION

Practice range queries over sorted points with insertion and deletion.
Write the K-generalization in words before coding: identify the local candidate window, test the predicate, then remove exactly the emitted points.
Rehearse explaining float comparison, tie-breaking, and what to do when multiple groups qualify.

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Sorting

MEDIUM | PracHub | Technical Screen | 9 of 12

★ TOP 75: rank 16 in the Medium and Hard priority list

★ Find Containing Range

POSTED 2026-06-04 | 138 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 65.2** | **30 DAYS 41.1** | **PRIORITY 12/228**

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Sorting

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #56 Merge Intervals

LeetCode match: No direct match; related LC 56 Merge Intervals + binary search (inferred from overview)

Posted on PracHub: 2026-06-04 (updated 2026-06-06)

Original: https://prachub.com/coding-questions/find-containing-range

QUESTION

Overview: This question evaluates the ability to design appropriate data structures and apply algorithmic reasoning for efficient interval containment queries, emphasizing preprocessing, correctness, and time-space trade-offs.

APPROACH

Sort/merge ranges once, then answer each query with binary search on starts and check the end. O(n log n) preprocessing, O(log n) per query.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the given ranges can overlap, what to return when a point lies in several ranges (the original range, the merged range, or all of them), and what to return when none contains it.

BRUTE FORCE TO SAY OUT LOUD

For each query scan all ranges. O(n) per query.

OPTIMAL IDEA

Sort the ranges by start and merge overlaps once, O(n log n). For each query binary search for the last merged range whose start is at most the point, then check that the point is at most its end. O(log n) per query. If the original (unmerged) range is required and ranges may overlap, use an interval tree or keep the maximum end seen in a prefix array.

DRY-RUN THESE CASES

A point before all ranges, after all ranges, in a gap, exactly on an endpoint, inside two overlapping ranges.

LIKELY FOLLOW-UPS

Insert and delete ranges (a balanced tree), or many point queries at once (sort and sweep).

PITFALLS

INTERVIEW-DAY PLAN

Binary searching without merging when ranges overlap. Mishandling the endpoint rule.

HOW TO THINK ABOUT IT

Merge the ranges once so they are disjoint and sorted; then 'which range contains x' is 'the last range whose start is at most x, and does its end reach x'. That is one binary search per query. Be clear whether the answer is the original range or the merged one, since they differ when ranges overlap.

Basis: Written (compare LeetCode 56 and a binary search for the range). Assumed rule (confirm it): given possibly overlapping ranges, answer queries 'which merged range contains x?', or None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def find_containing(ranges, x):
    for start, end in ranges:
        if start <= x <= end:
            return (start, end)
    return None
```

TIME	SPACE
O(n) per query	**O(n)**

WHY IT IS SLOW

Every query walks all n ranges. Merging once (O(n log n)) and binary searching the sorted starts makes each query O(log n).

OPTIMAL SOLUTION · Merge once, then binary search the starts

TIME	SPACE
O(n log n) to build, O(log n) per query	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return every original range that contains x, not the merged one.

Sort by start and consider only ranges starting at or before x (one bisect); among those, keep the ones whose end is at least x. With many overlaps use an interval tree for O(log n + k) per query.

FOLLOW-UP 2 Ranges are inserted and deleted over time.

Keep the disjoint ranges in a sorted container (balanced tree); an insert merges neighbours, and a query is the same predecessor search. O(log n) per operation.

FOLLOW-UP 3 Queries are given in bulk.

Sort the queries and sweep once over the merged ranges with two pointers: O((n + q) log) overall instead of a binary search per query.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find Containing Range
#
# This question evaluates the ability to design
# appropriate data structures and apply algorithmic
# reasoning for efficient interval containment queries,
# emphasizing preprocessing, correctness, and time-space
# trade-offs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return every original range that contains x, not the
# merged one.
# 2. Ranges are inserted and deleted over time.
# 3. Queries are given in bulk.
#
# INTERVIEW NOTES
# Merge the ranges once so they are disjoint and sorted;
# then 'which range contains x' is 'the last range whose
# start is at most x, and does its end reach x'. That is
# one binary search per query. Be clear whether the answer
# is the original range or the merged one, since they
# differ when ranges overlap.
#
# BRUTE FORCE (say it first): Scan all ranges for each
# query
# Merge the ranges (or not) and for each query test every
# range for containment. Correct and the oracle; each
# query walks the whole list. Time O(n) per query. Space
# O(n).


from bisect import bisect_left, bisect_right


# BRUTE FORCE (say it first): Scan all ranges for each
# query | time O(n) per query
def containing_range_scan(ranges, x):
    merged = []
    for s, e in sorted(ranges):
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    for s, e in merged:
        if s <= x <= e:
            return (s, e)
    return None


# BASE QUESTION (optimal): Merge once, then binary search
# the starts
# Idea: After sorting and merging, the starts are
# increasing, so bisect_right(starts, x) - 1 is the only
# candidate; it contains x if its end is at least x.
# Preprocessing is paid once.
# FOLLOW-UP 1: Return every original range that contains
# x, not the merged one.
# Answer: Sort by start and consider only ranges starting
# at or before x (one bisect); among those, keep the ones
# whose end is at least x. With many overlaps use an
# interval tree for O(log n + k) per query.
class RangeIndex:
    """Merged, sorted, disjoint closed ranges."""

    def __init__(self, ranges=()):
        self.starts, self.ends = [], []
        self.original = sorted(ranges)
        for s, e in self.original:
            self.insert(s, e)

    # FOLLOW-UP 2: ranges can be inserted over time
    def insert(self, s, e):
        # first range ending at or after s ...
        i = bisect_left(self.ends, s)
        # ... up to the last range starting at or before e
        j = bisect_right(self.starts, e)
        if i < j:
            s = min(s, self.starts[i])
            e = max(e, self.ends[j - 1])
        # replace everything the new range touches
        self.starts[i:j] = [s]
        self.ends[i:j] = [e]

    def find(self, x):
        # the merged range containing x, or None
        i = bisect_right(self.starts, x) - 1
        if i >= 0 and self.ends[i] >= x:
            return self.starts[i], self.ends[i]
        return None

    # FOLLOW-UP 1: every ORIGINAL range containing x
    def find_original(self, x):
        # every ORIGINAL range that contains x
        k = bisect_right([s for s, _ in self.original], x)
        return [r for r in self.original[:k] if r[1] >= x]

    # FOLLOW-UP 3: queries given in bulk
    def find_many(self, queries):
        # bulk: sort the queries and sweep once
        order = sorted(range(len(queries)),
                       key=lambda i: queries[i])
        out, j = [None] * len(queries), 0
        for i in order:
            x = queries[i]
            # skip ranges that end before x
            while j < len(self.ends) and self.ends[j] < x:
                j += 1
            inside = j < len(self.starts)
            if inside and self.starts[j] <= x:
                out[i] = (self.starts[j], self.ends[j])
        return out


if __name__ == "__main__":
    idx = RangeIndex([(1, 3), (2, 6), (8, 10), (15, 18)])
    assert idx.find(4) == (1, 6)
    assert idx.find(7) is None
    assert idx.find(8) == (8, 10)
    assert idx.find(0) is None and idx.find(99) is None
    # the original ranges that cover 2
    assert idx.find_original(2) == [(1, 3), (2, 6)]
    # touching ranges merge: (6, 8) joins the first two
    # blocks
    idx.insert(6, 8)
    assert idx.find(7) == (1, 10)
    assert idx.find(12) is None
    qs = [4, 20, 16, 9]
    assert idx.find_many(qs) == [idx.find(q) for q in qs]
```

FUNCTION AND TEST CASES

Signature

```
def solution(ranges: list[list[int]], queries: list[int])
-> list[list[int]]:
```

Sample call

```
solution([[1, 3], [50, 100]], [2, 50, 10])
```

Test cases

Input	Expected
`([[1, 3], [50, 100]], [2, 50, 10])`	`[[1, 3], [50, 100], None]`
`([[5, 8], [-10, -5], [0, 0]], [-7, 0, 8, 9, -11])`	`[[-10, -5], [0, 0], [5, 8], None, None]`

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Sorting

MEDIUM | PracHub | Technical Screen | 10 of 12

★ TOP 75: rank 22 in the Medium and Hard priority list

★ Check if all substrings are anagrams of words

POSTED 2026-03-11 | 223 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 51.4 | PRIORITY 16/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Sorting

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #49 Group Anagrams

LeetCode match: No direct match; related LC 49 Group Anagrams

Posted on PracHub: 2026-03-11 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/check-if-all-substrings-are-anagrams-of-words

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** can be **rearranged (anagrammed)** into a valid dictionary word. Formally, for every substring **`t = s[i..j]`** with **`|t| >= 3`**, there exists some word **`w in dict`** such that **`w`** is a permutation of **`t`** (same multiset of characters). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**. - "Can be rearranged into a word" means the substring and the dictionary word have exactly the same character counts. Example:
- If **`t = "act"`**, it passes if the dictionary contains any of **`{ "act", "cat", "tac" }`**.

APPROACH

Precompute sorted-letter (or count-vector) signatures of dictionary words into a set; for each substring of length >= 3 compute its signature and look it up.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether every substring of length at least 3 must be rearrangeable into some dictionary word, the alphabet (letters only), and the dictionary size and word lengths.

BRUTE FORCE TO SAY OUT LOUD

For every substring, compare against every dictionary word by sorted letters. Very slow.

OPTIMAL IDEA

INTERVIEW-DAY PLAN

Preprocess the dictionary into a set of signatures, where a signature is the sorted letters or a 26-count tuple, and also record the set of word lengths present. For each start i extend j one letter at a time, updating a count vector incrementally; once the length is at least 3, look up its signature (skip lengths that no word has) and return false at the first miss. O(n^2 * 26) time.

DRY-RUN THESE CASES

A string shorter than 3, a failing length that has no dictionary words, a long repeated letter, all substrings valid.

LIKELY FOLLOW-UPS

Return the failing substring, or a larger alphabet (use a hash of counts).

PITFALLS

Sorting each substring from scratch, which costs an extra log factor. Forgetting that a signature must match the length too (a count tuple already does).

HOW TO THINK ABOUT IT

Anagrams share a signature: the multiset of letters (a sorted string or a 26-count tuple). So turn the dictionary into a set of signatures once, and test each substring by its signature. Extending a substring by one letter changes its count vector in O(1), so you do not rebuild the signature from scratch.

Basis: Written; every substring of length >= 3 must be a permutation of some dictionary word (compare Group Anagrams, LeetCode 49).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, words):
    for i in range(len(s)):
        for j in range(i + 3, len(s) + 1):
            key = sorted(s[i:j])
            match = [w for w in words
                     if key == sorted(w)]
            if not match:
                return False
    return True
```

TIME	SPACE
O(n^2 * W * L log L)	O(1) extra

WHY IT IS SLOW

Each substring is compared with every word and sorted again: O(n^2 * W * L log L). A set of word signatures, with the substring's count vector updated incrementally, makes each check O(alphabet).

OPTIMAL SOLUTION · Set of signatures, incremental count vectors

TIME	SPACE
O(n^2 * A) for alphabet size A	O(dictionary size)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A bigger alphabet or Unicode.

Replace the fixed array by a Counter and use a frozen tuple of sorted (letter, count) pairs, or a hash of the counts, as the signature. **Cost:** O(n^2 * distinct letters in a substring) with a Counter signature instead of a fixed array.

FOLLOW-UP 2 Only substrings of lengths that exist in the dictionary need checking.

Record the set of word lengths; a substring whose length is not in that set can never match, so you can return False the first time such a length is reached (assuming all substrings of length >= 3 up to n must match).

FOLLOW-UP 3 Subsets of the letters (any sub-multiset of a word) instead of a full permutation.

Then the test is containment of count vectors, which is a subset-of-multiset query; use a sorted signature trie or precompute all sub-multisets for short words.

FOLLOW-UP 4 Report the first failing substring instead of True or False.

Test substrings by length (shortest first) so the report is a smallest counterexample; the signature set makes each test one lookup after sorting the substring's letters.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check if all substrings are anagrams of
# words
#
# You are given a string s (letters only) and access to an
# English dictionary dict (a set of valid words). Return
# true if **every contiguous substring** of s with length
# **>= 3** can be **rearranged (anagrammed)** into a valid
# dictionary word. Formally, for every substring t =
# s[i..j] with |t| >= 3, there exists some word w in dict
# such that w is a permutation of t (same multiset of
# characters). Otherwise return false. Clarifications: -
# Substring means **contiguous**. - “Can be rearranged
# into a word” means the substring and the dictionary word
# have exactly the same character counts. Example: - If t
# = "act", it passes if the dictionary contains any of {
# "act", "cat", "tac" }.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A bigger alphabet or Unicode.
# 2. Only substrings of lengths that exist in the
# dictionary need checking.
# 3. Subsets of the letters (any sub-multiset of a word)
# instead of a full permutation.
# 4. Report the first failing substring instead of True or
# False.
#
# INTERVIEW NOTES
# Anagrams share a signature: the multiset of letters (a
# sorted string or a 26-count tuple). So turn the
# dictionary into a set of signatures once, and test each
# substring by its signature. Extending a substring by one
# letter changes its count vector in O(1), so you do not
# rebuild the signature from scratch.
#
# BRUTE FORCE (say it first): Compare each substring with
# every word's sorted letters
# For each substring, sort its letters and compare with
# the sorted letters of every dictionary word. Correct,
# and the dictionary is rescanned for each substring. Time
# O(n^2 * W * L log L). Space O(1) extra.


from collections import Counter
```

```
# BRUTE FORCE (say it first): Compare each substring with
# every word's sorted letters | time O(n^2 * W * L log L)
def _sig(w):
    return tuple(sorted(w))


def all_anagram_brute(s, words):
    sigs = [_sig(w) for w in words]
    return all(
        _sig(s[i:j]) in sigs
        for i in range(len(s))
        for j in range(i + 3, len(s) + 1)
    )


# HELPERS for the base question
ALPHABET = "abcdefghijklmnopqrstuvwxyz"


# HELPERS for follow-up 4
def signature(text):
    # the letters in sorted order: equal for anagrams
    return "".join(sorted(text))


# BASE QUESTION (optimal): Set of signatures, incremental
# count vectors
# Idea: Precompute the set of count-vector signatures of
# the dictionary words. For each start extend one letter
# at a time, update the count vector in O(1), and look up
# its tuple (once the length is at least 3).
def all_anagram_words(s, words, min_len=3):
    # every substring needs a dictionary word with the
    # same
    # letters. Count vectors are updated as the window
    # grows.
    index = {c: i for i, c in enumerate(ALPHABET)}
    sigs = set()
    for w in words:
        v = [0] * len(ALPHABET)
        for ch in w:
            v[index[ch]] += 1
        sigs.add(tuple(v))
    for i in range(len(s)):
```

```
        v = [0] * len(ALPHABET)
```

```
        for j in range(i, len(s)):
            v[index[s[j]]] += 1
            long_enough = j - i + 1 >= min_len
            if long_enough and tuple(v) not in sigs:
                return False
    return True


# FOLLOW-UP 1: A bigger alphabet or Unicode.
# Answer: Replace the fixed array by a Counter and use a
# frozen tuple of sorted (letter, count) pairs, or a hash
# of the counts, as the signature.
def all_anagram_words_any_alphabet(s, words, min_len=3):
    # Unicode or a big alphabet: a frozen Counter as
    # signature
    sigs = {frozenset(Counter(w).items()) for w in words}
    for i in range(len(s)):
        count = Counter()
        for j in range(i, len(s)):
            count[s[j]] += 1
            key = frozenset(count.items())
            if j - i + 1 >= min_len and key not in sigs:
                return False
    return True


# FOLLOW-UP 4: Report the first failing substring instead
# of True or False.
# Answer: Test substrings by length (shortest first) so
# the report is a smallest counterexample; the signature
# set makes each test one lookup after sorting the
# substring's letters.
def first_failing(s, words, min_len=3):
    # smallest counterexample: lengths from short to long
    sigs = {signature(w) for w in words}
    lengths = {len(w) for w in words}
    for length in range(min_len, len(s) + 1):
        # no word has this length: the first window fails
        if length not in lengths:
            return s[:length]
        for i in range(len(s) - length + 1):
            piece = s[i:i + length]
            if signature(piece) not in sigs:
                return piece
    return None
```

```
if __name__ == "__main__":
    words = {"cat", "act", "tac", "tact", "cats"}
    assert all_anagram_words("cat", words)
    assert all_anagram_words("tca", words)
    assert not all_anagram_words("tcaz", words)
    assert all_anagram_words_any_alphabet("cat", words)
    assert first_failing("cat", words) is None
    # "catz" fails at length 3? "atz" is not an anagram
    assert first_failing("catz", words) == "atz"
    assert all_anagram_words("ab", set())
    assert first_failing("aaaa", {"aaa"}) == "aaaa"
```

FUNCTION AND TEST CASES

Signature

```
def solution(s: str, words: list[str]) -> bool:
```

Sample call

```
solution("act", ["cat", "dog"])
```

Test cases

Input	Expected
("act", ["cat", "dog"])	True
("abca", ["cab", "abca", "zzz"])	True

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Sorting

MEDIUM | PracHub | Technical Screen | 11 of 12

★ TOP 75: rank 35 in the Medium and Hard priority list

★ Find shared objects across two log files

POSTED 2025-11-05 | 349 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 50.4 | 30 DAYS 46.9 | PRIORITY 23/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Sorting

Optimal answer from: the interview site's own solution

LeetCode match: LC 349 Intersection of Two Arrays

Posted on PracHub: 2025-11-05 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/find-shared-objects-across-two-log-files

QUESTION

You are given two large log files representing activity on two different days. Each line of each log has three fields:

- **`timestamp`**: a time value (you may treat it as an opaque string or integer)
- **`obj_id`**: identifier of an object (string or integer)
- **`client_id`**: identifier of a client (string or integer) The logs are not necessarily sorted by any field. Define an object as **interesting** if:

1. Its **`obj_id`** appears at least once in day 1's log **and** at least once in day 2's log; and 2. Across all of its appearances in both days combined, it is associated with at least **two distinct `client_id`** values.

Tasks

1. Design and implement a function that, given file handles (or streaming iterators) for the two log files, returns the set of all such interesting **`obj_id`**s.
2. For your initial, straightforward in-memory solution (you may assume both logs fit in memory), state the time and space complexity in terms of the total number of log records.
3. Follow-up (memory constrained): Now assume the logs are very large and **cannot** both fit into memory at once.

- You may still sequentially scan each file and you may perform external sorting.
- Describe how you would modify your approach to work under this memory constraint.
- Analyze the time and space complexity of this memory-efficient solution.

Return sorted obj_ids that appear in both day logs and have at least two distinct client_ids across both days combined.
Constraints
Rows are [timestamp, obj_id, client_id] or dicts with those fields

QUESTION

EXAMPLES

```
Input: ([['t1', 'o1', 'c1'], ['t2', 'o2', 'c1']],
    [['t3', 'o1', 'c2'], ['t4', 'o2', 'c1']])
Expected Output: ['o1']
Input: ([], [['t', 'o', 'c']])
Expected Output: []
```

Hints

Track which days each object appears in and its set of clients.

APPROACH

Load the smaller file's obj_ids into a hash set, stream the other and collect matches; if both are huge use external sort/merge or hash partitioning to disk (or a Bloom filter prefilter).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether to report distinct ids or every occurrence, how large each file is, whether order matters, and what to output (ids only or full lines).

BRUTE FORCE TO SAY OUT LOUD

For each id in file A scan file B. O(n * m), unusable at scale.

OPTIMAL IDEA

Load the ids of the smaller file into a hash set, stream the larger file and collect hits (O(n + m) time). If both do not fit in memory, hash-partition both files by id into buckets on disk so each bucket pair fits, then join bucket by bucket; or external sort both and merge. A Bloom filter of one file prefilters the other.

DRY-RUN THESE CASES

Empty file, duplicate ids, identical files, no overlap, ids that differ only in case or whitespace.

LIKELY FOLLOW-UPS

Distributed version (shuffle by key), top N shared ids by frequency, restrict to a time window.

PITFALLS

Loading both files into memory. Not deduplicating output.

HOW TO THINK ABOUT IT

A common key in two files is a join. If one file fits in memory, load its keys into a set and stream the other. If neither fits, split both files by hash of the key into buckets small enough for memory; equal keys always land in the same bucket number, so you only compare bucket i of one file with bucket i of the other (Grace hash join).

Basis: Written; a join of two large relations (external hash join). Line format assumed: 'timestamp obj_id other'.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(file_a, file_b):
    shared = set()
    for a in file_a:
        for b in file_b:
            if a.obj_id == b.obj_id:
                shared.add(a.obj_id)
    return shared
```

TIME	SPACE
O(n * m)	O(1) extra

WHY IT IS SLOW

O(n * m) comparisons. Loading the smaller file's ids into a hash set and streaming the other file is O(n + m); if neither fits, partition both by a hash of the id.

OPTIMAL SOLUTION · Hash set of the smaller file's ids

TIME	SPACE
O(n + m)	O(distinct ids of the smaller file)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Both files are too big for memory.

Hash-partition both files into B buckets on disk by hash of the object id, then process bucket pairs one at a time with the in-memory set join. Choose B so that one bucket of the smaller file fits in memory. External sort and merge also works.

FOLLOW-UP 2 Want objects seen on both days within an hour of each other.

Keep a list of timestamps per id in the set join and compare timestamps (sorted lists with two pointers), instead of just membership.

FOLLOW-UP 3 A tiny memory budget and an approximate answer is fine.

Build a Bloom filter from one file and test the other against it, accepting a small false-positive rate; verify the few hits exactly in a second pass if needed. **Cost:** O(n + m) time with O(bits) memory for the Bloom filter and a small false-positive rate.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find shared objects across two log files
#
# You are given two large log files representing activity
# on two different days. Each line of each log has three
# fields: - timestamp: a time value (you may treat it as
# an opaque string or integer) - obj_id: identifier of an
# object (string or integer) - client_id: identifier of a
# client (string or integer) The logs are not necessarily
# sorted by any field. Define an object as **interesting**
# if: 1. Its obj_id appears at least once in day 1's log
# **and** at least once in day 2's log; and 2. Across all
# of its appearances in both days combined, it is
# associated with at least **two distinct** client_id
# values. ### Tasks 1. Design and implement a function
# that, given file handles (or streaming iterators) for
# the two log files, returns the set of all such
# interesting obj_ids. 2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Both files are too big for memory.
# 2. Want objects seen on both days within an hour of each
# other.
# 3. A tiny memory budget and an approximate answer is
# fine.
#
# INTERVIEW NOTES
# A common key in two files is a join. If one file fits in
# memory, load its keys into a set and stream the other.
# If neither fits, split both files by hash of the key
# into buckets small enough for memory; equal keys always
# land in the same bucket number, so you only compare
# bucket i of one file with bucket i of the other (Grace
# hash join).
#
# BRUTE FORCE (say it first): Compare every pair of lines
# For each line in A scan all of B for the same object id.
# Correct, but quadratic in the file sizes. Time O(n * m).
# Space O(1) extra.


import zlib
from collections import defaultdict


# BRUTE FORCE (say it first): Compare every pair of lines
```

```
# | time O(n * m)
def shared_brute(a_lines, b_lines):
    out = set()
    for x in a_lines:
        for y in b_lines:
            if x.split()[1] == y.split()[1]:
                out.add(x.split()[1])
    return out


# HELPERS for follow-up 1
def bucket_of(obj, buckets):
    # a stable hash (Python's own hash changes between
    # runs)
    return zlib.crc32(str(obj).encode()) % buckets


# BASE QUESTION (optimal): Hash set of the smaller file's
# ids
# Idea: Load the object ids of the smaller file into a
```

```python
# set, then stream the other file and keep ids that are in
# the set. Each line is read once; memory depends only on
# the smaller file.
def shared_objects(day1, day2):
    # rows are [timestamp, obj_id, client_id]. Remember,
    # per
    # object, which days it appeared on and its clients.
    days, clients = defaultdict(set), defaultdict(set)
    for number, rows in ((1, day1), (2, day2)):
        for _, obj, client in rows:
            days[obj].add(number)
            clients[obj].add(client)
    return sorted(o for o in days
                  if len(days[o]) == 2
                  and len(clients[o]) >= 2)


# OTHER WAY: external sort of both days
def shared_objects_sorted(day1, day2):
    # the other way: external sort both days by object,
    # then a
    # merge that holds only one object's clients at a time
    a = sorted(day1, key=lambda r: r[1])
    b = sorted(day2, key=lambda r: r[1])
    i = j = 0
```

```python
    out = []
    while i < len(a) and j < len(b):
        if a[i][1] < b[j][1]:
            i += 1
        elif a[i][1] > b[j][1]:
            j += 1
        else:
            obj, clients = a[i][1], set()
            while i < len(a) and a[i][1] == obj:
                clients.add(a[i][2])
                i += 1
            while j < len(b) and b[j][1] == obj:
                clients.add(b[j][2])
                j += 1
            if len(clients) >= 2:
                out.append(obj)
    return out


# FOLLOW-UP 1: Both files are too big for memory.
# Answer: Hash-partition both files into B buckets on disk
# by hash of the object id, then process bucket pairs one
# at a time with the in-memory set join. Choose B so that
# one bucket of the smaller file fits in memory.
def shared_objects_partitioned(day1, day2, buckets=4):
    # memory limit: write each row to a bucket "file"
    # chosen by
    # the hash of its object, then solve one bucket at a
    # time.
    # All rows of an object land in the same bucket.
    parts1 = [[] for _ in range(buckets)]
    parts2 = [[] for _ in range(buckets)]
    for rows, parts in ((day1, parts1), (day2, parts2)):
        for row in rows:
            parts[bucket_of(row[1], buckets)].append(row)
    out = []
    for b in range(buckets):
        out += shared_objects(parts1[b], parts2[b])
    return sorted(out)


# FOLLOW-UP 3: A tiny memory budget and an approximate
# answer is fine.
# Answer: Build a Bloom filter from one file and test the
# other against it, accepting a small false-positive rate;
```

```python
# verify the few hits exactly in a second pass if needed.
class Bloom:
    """Tiny memory, approximate: may say 'yes' for a few
    objects that were never added."""

    def __init__(self, bits=1 << 16, hashes=3):
        self.bits, self.hashes = bytearray(bits), hashes

    def spots(self, key):
        for seed in range(self.hashes):
            text = f"{seed}:{key}".encode()
            yield zlib.crc32(text) % len(self.bits)

    def add(self, key):
        for s in self.spots(key):
            self.bits[s] = 1

    def maybe(self, key):
        return all(self.bits[s] for s in self.spots(key))


if __name__ == "__main__":
    d1 = [["t1", "o1", "c1"], ["t2", "o2", "c1"]]
    d2 = [["t3", "o1", "c2"], ["t4", "o2", "c1"]]
    assert shared_objects(d1, d2) == ["o1"]
    assert shared_objects([], [["t", "o", "c"]]) == []
    assert shared_objects_partitioned(d1, d2, 3) == ["o1"]
    assert shared_objects_sorted(d1, d2) == ["o1"]
    many1 = [[i, f"o{i % 7}", f"c{i % 3}"]
             for i in range(40)]
    many2 = [[i, f"o{i % 5}", f"c{i % 2}"]
             for i in range(40)]
    want = shared_objects(many1, many2)
    parts = shared_objects_partitioned(many1, many2, 5)
    assert parts == want
    assert shared_objects_sorted(many1, many2) == want
    bloom = Bloom()
    for row in d1:
        bloom.add(row[1])
    assert bloom.maybe("o1") and not bloom.maybe("o999")
```

ALSO REPORTED · 1 more

Two-Day Log Intersection under Memory Limit | 1Point3Acres | onsite-coding | asked 2025-11-27 | Single report

https://www.1point3acres.com/interview/problems/company/google/two-logs-intersection-memory-limit

In-memory solution: build a hashmap obj_id -> set(client_id) while scanning each file; intersect keys across the two hashmaps at the end.

Memory-limited solution: external sort each file by obj_id, then run a merge / two-pointer scan over both sorted files. For each matching obj_id group, count distinct client_id on the fly (HyperLogLog if duplicates are large).

The interviewer explicitly preferred sort + two-pointer over a streaming hashmap — be prepared to defend the choice (sort is O(n log n) on disk vs O(n) RAM-bound for the hashmap).

For distributed scale, hash-partition by obj_id and run the same algorithm per partition.

ALSO REPORTED · 1 more

Preparation: Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Sorting

HARD | PracHub | Technical Screen | 12 of 12

★ TOP 75: rank 10 in the Medium and Hard priority list

★ Find safe travel intervals between planet influences

POSTED 2025-11-22 | 332 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 65.2 | 30 DAYS 41.1 | PRIORITY 9/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Sorting

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #56 Merge Intervals

LeetCode match: LC 56 Merge Intervals (then take gaps)

Posted on PracHub: 2025-11-22 (updated 2026-05-23)

Original: https://prachub.com/coding-questions/find-safe-travel-intervals-between-planet-influences

QUESTION

You are planning a space route along a one-dimensional line (the x-axis). You are given a list of planets. Each planet is represented by an integer pair ((c, r)):

- **c** is the coordinate of the planet's center on the x-axis.
- **r** is a non-negative integer radius of its gravitational influence. A point **x** on the x-axis is **unsafe** for a space traveler if it lies within the gravitational influence of at least one planet: [|x - c_i| \le r_i \quad \text{for some planet } i] Equivalently, each planet makes the interval ([c_i - r_i,\; c_i + r_i]) unsafe. Task:
- Consider the entire real line as the possible travel path.
- Compute all **maximal continuous intervals** on this line where the traveler is **safe**, i.e., points not covered by any planet's unsafe interval.
- Return these safe intervals as a sorted list of disjoint intervals. Representation details:
- Represent each safe interval as a pair **`(start, end)`** with **`start < end`**.
- Intervals may be unbounded on the left or right:
- Use **`-inf`** for an interval unbounded to the left (e.g., **`(-inf, a)`**).
- Use **`+inf`** for an interval unbounded to the right (e.g., **`(b, +inf)`**). You may assume the input list of planets can be in any order and may contain overlapping or nested gravitational ranges. Return the safe intervals in increasing order of **`start`**.

QUESTION

You are planning a route along the real x-axis. Each planet is given as a pair (c, r), where c is its center and r is a non-negative radius of gravitational influence. A point x is unsafe if it lies inside at least one planet's influence, meaning |x - c| <= r. So each planet creates an unsafe closed interval [c - r, c + r]. Your task is to return all maximal continuous safe intervals on the real line: regions not covered by any unsafe interval. Represent each safe interval as a tuple (start, end), where the interval means start < x < end. Because JSON/Python literals do not have a direct infinity literal, use None to represent an unbounded side:

- (None, a) means (-infinity, a)
- (b, None) means (b, +infinity)
- (None, None) means the entire real line Return the safe intervals in increasing order. Overlapping, nested, and touching unsafe intervals should be treated as one merged unsafe region.

Constraints
0 <= len(planets) <= 200000-10^9 <= c <= 10^90 <= r <= 10^9

EXAMPLES

```
Input: []
Expected Output: [(None, None)]
```

Explanation: With no planets, no point is unsafe, so the entire real line is safe.

```
Input: [(3, 1), (10, 2)]
Expected Output: [(None, 2), (4, 8), (12, None)]
Explanation: The unsafe intervals are [2, 4] and [8,
    12]. Their complement is (-infinity, 2), (4, 8),
    and (12, +infinity).
```

Hints

First convert every planet (c, r) into the unsafe interval [c - r, c + r].Sort the unsafe intervals and merge all overlapping or touching ones. The safe intervals are the gaps before, between, and after the merged intervals.

APPROACH

Turn each planet into [c-r, c+r], sort, merge overlaps, and the safe intervals are the open gaps between merged intervals plus the two infinite ends. Watch endpoint inclusivity (unsafe intervals are closed).

HOW TO THINK ABOUT IT

The safe points are the complement of the union of the unsafe intervals. Merge the unsafe intervals first (sort by start, extend while they overlap or touch), and the safe intervals are exactly the gaps between consecutive merged intervals plus the two unbounded ends. Endpoints themselves are unsafe because the influence is closed (<=).

Basis: Interval complement (LeetCode 56, Merge Intervals, then take the gaps). Each planet (c, r) makes the closed interval [c - r, c + r] unsafe; safe intervals are the open gaps, possibly unbounded.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(planets):
    pts = set()
    for c, r in planets:
        pts |= {c - r, c + r}
    bounds = [-INF] + sorted(pts) + [INF]
    safe = []
    for a, b in zip(bounds, bounds[1:]):
        mid = midpoint(a, b)
        if not any(abs(mid - c) < r
                   for c, r in planets):
            safe.append((a, b))
    return safe
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each midpoint is tested against every planet, O(n^2). Merging the unsafe intervals after sorting and reading the gaps is O(n log n).

OPTIMAL SOLUTION · Merge the unsafe intervals, then read off the gaps

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the part between two given positions matters (a route from A to B).

Intersect each safe interval with [A, B] and keep the non-empty ones. Or clip the planets first; the merge-and-gap logic is the same.

FOLLOW-UP 2 Are the endpoints safe?

The statement uses <=, so the endpoints of an unsafe interval are unsafe and the safe intervals are open. If the influence were strict (<), touching intervals would leave a single safe point between them and the merge test would use < instead of <=.

FOLLOW-UP 3 Planets arrive over time.

Keep the merged unsafe intervals in a sorted structure; an insert merges neighbours (like the stream-of-intervals problem) and the safe intervals are always the gaps.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find safe travel intervals between planet
# influences
#
# You are planning a space route along a one-dimensional
# line (the x-axis). You are given a list of planets. Each
# planet is represented by an integer pair \((c, r)\): - c
# is the coordinate of the planet's center on the x-axis.
# - r is a non-negative integer radius of its
# gravitational influence. A point x on the x-axis is
# **unsafe** for a space traveler if it lies within the
# gravitational influence of at least one planet: \[ |x -
# c_i| \le r_i \quad \text{for some planet } i \]
# Equivalently, each planet makes the interval \([c_i -
# r_i,\; c_i + r_i]\) unsafe. Task: - Consider the entire
# real line as the possible travel path. - Compute all
# **maximal continuous intervals** on this line where the
# traveler is **safe**, i.e., points not covered by any
# planet's unsafe interval. - Return these safe intervals
# as a sorted list of disjoint intervals.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the part between two given positions matters (a
# route from A to B).
# 2. Are the endpoints safe?
# 3. Planets arrive over time.
#
# INTERVIEW NOTES
# The safe points are the complement of the union of the
# unsafe intervals. Merge the unsafe intervals first (sort
# by start, extend while they overlap or touch), and the
# safe intervals are exactly the gaps between consecutive
# merged intervals plus the two unbounded ends. Endpoints
# themselves are unsafe because the influence is closed
# (<=).
#
# BRUTE FORCE (say it first): Test the midpoint of every
# segment between critical points
# Collect all interval endpoints, and for every
# consecutive pair (and the two outer rays) test whether
# the midpoint is covered by some planet. Adjacent safe
# segments are separate because their shared endpoint is
# unsafe. Correct and the oracle; every midpoint is tested
# against every planet. Time O(n^2). Space O(n).
```

```
# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first): Test the midpoint of every
# segment between critical points | time O(n^2)
def safe_intervals_brute(planets):
    iv = [(c - r, c + r) for c, r in planets]
    pts = sorted({x for a, b in iv for x in (a, b)})
    if not pts:
        return [(-INF, INF)]

    def safe(x):
        return not any(a <= x <= b for a, b in iv)

    out = []
    if safe(pts[0] - 1):
        out.append((-INF, pts[0]))
    for p, q in zip(pts, pts[1:]):
        if safe((p + q) / 2):
            out.append((p, q))
```

```
    if safe(pts[-1] + 1):
        out.append((pts[-1], INF))
    return out


# HELPERS for the base question
def unsafe_blocks(planets):
    # planet (c, r) makes [c - r, c + r] unsafe; merge
    # overlaps
    merged = []
    spans = sorted((c - r, c + r) for c, r in planets)
    for a, b in spans:
        # sharing even one point joins two blocks
        if merged and a <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], b)
        else:
            merged.append([a, b])
    return merged


# HELPERS for follow-up 3
def safe_intervals_from(blocks):
    if not blocks:
        return [(-INF, INF)]
```

```
    out = [(-INF, blocks[0][0])]
    for prev, nxt in zip(blocks, blocks[1:]):
        out.append((prev[1], nxt[0]))
    out.append((blocks[-1][1], INF))
    return out


# BASE QUESTION (optimal): Merge the unsafe intervals,
# then read off the gaps
# Idea: Sort the unsafe intervals by start and merge any
# that overlap or touch (they share a point, which is
# unsafe).
def safe_intervals(planets):
    # the gaps between unsafe blocks (open intervals)
    merged = unsafe_blocks(planets)
    if not merged:
        return [(-INF, INF)]
    out = [(-INF, merged[0][0])]
    for prev, nxt in zip(merged, merged[1:]):
        out.append((prev[1], nxt[0]))
    out.append((merged[-1][1], INF))
    return out


# FOLLOW-UP 1: Only the part between two given positions
# matters (a route from A to B).
# Answer: Intersect each safe interval with [A, B] and
# keep the non-empty ones. Or clip the planets first; the
# merge-and-gap logic is the same.
def safe_between(planets, a, b):
    # a route from a to b: clip every safe interval to [a,
    # b]
    out = []
    for lo, hi in safe_intervals(planets):
        lo, hi = max(lo, a), min(hi, b)
        if lo < hi:
            out.append((lo, hi))
    return out


# FOLLOW-UP 3: Planets arrive over time.
# Answer: Keep the merged unsafe intervals in a sorted
# structure; an insert merges neighbours (like the stream-
# of-intervals problem) and the safe intervals are always
# the gaps.
```

```
class LiveRoute:
    """Planets arrive over time: keep blocks merged."""
    def __init__(self):
        self.blocks = []

    def add(self, c, r):
        a, b = c - r, c + r
        keep = []
        for s, e in self.blocks:
            if e < a or s > b:
                keep.append([s, e])
            else:
                # overlaps the new block: absorb it
                a, b = min(a, s), max(b, e)
        keep.append([a, b])
        self.blocks = sorted(keep)

    def safe(self):
        return safe_intervals_from(self.blocks)


if __name__ == "__main__":
    planets = [(0, 2), (10, 1), (3, 1)]
    # unsafe: [-2, 4] and [9, 11]
    assert safe_intervals(planets) == [(-INF, -2), (4, 9),
                                       (11, INF)]
    assert safe_intervals([]) == [(-INF, INF)]
    # touching blocks merge: [0, 2] and [2, 4]
    assert safe_intervals([(1, 1), (3, 1)]) == [
        (-INF, 0), (4, INF)]
    # only the part of the route between 5 and 20 matters
    route = safe_between(planets, 5, 20)
    assert route == [(5, 9), (11, 20)]
    live = LiveRoute()
    for c, r in planets:
        live.add(c, r)
    assert live.safe() == safe_intervals(planets)
    # a big planet at 6 swallows both blocks
    live.add(6, 3)
    assert live.safe() == [(-INF, -2), (11, INF)]
```

FUNCTION AND TEST CASES

Signature

```
def solution(planets: list[list[int]]) -> list[list[int]]:
SAMPLE INPUT
[]
```

Test cases

Input	Expected
`[]`	`[(None, None)]`
`[(3, 1), (10, 2)]`	`[(None, 2), (4, 8), (12, None)]`